

MBA EM **ENGENHARIA**
DE SOFTWARE

PATRONES DE DISEÑO (DESIGN PATTERNS) I

Profa. Dra. Rosana T. Vaccare Braga

MBAUSP
ESALQ

La responsabilidad por la idoneidad, originalidad
y licitud de los contenidos didácticos presentados es del profesor.

Prohibida la reproducción, total o parcial, sin autorización.

Ley nº 9610/98

Organización de la clase

- Reutilización de software y sus beneficios;
- Patrones y su origen;
- Patterns vs Standards;
- Elementos de un patrón, tipos de patrón;
- Patrones de diseño: definiciones, ventajas, categorías, formatos de escritura;
- Patrones GoF de la categoría “de creación” e inicio de la categoría “estructural”.



Reutilização de Software

- **Reutilização de Software:** replicación de cualquier tipo de conocimiento sobre un sistema en otros sistemas similares, con el objetivo de reducir el esfuerzo de desarrollo y el mantenimiento en esos nuevos sistemas (Biggerstaff y Perlis, 1989).
 - Años 70: módulos y subrutinas
 - Años 80: clases y generadores de aplicación
 - Años 90: análisis de dominio, componentes, patrones (*patterns*) y frameworks;
 - Años 2000: servicios, aspectos
 - ...

Importante:
patterns x standards

Reutilização de Software

- Benefícios:
 - Melhores índices de produtividade
 - Produtos de melhor qualidade, mais confiáveis, consistentes y estandarizados
 - Redução de los costos y el tiempo involucrados en el desarrollo de software
 - Mayor flexibilidad en la estructura del software producido, facilitando su mantenimiento y evolución

Importante: Patterns x Standards

Tema de este
módulo del MBA

Patterns:

- Proxy, Composite, Factory-method, etc. **(de diseño)**
- Capas, MVC, etc. (arquitectónicos)
- Transaction, Item-transaction, Type-object (de análisis)
- Site map, shopping cart, ... (de interacción con el usuario)
- etc...

Standards:

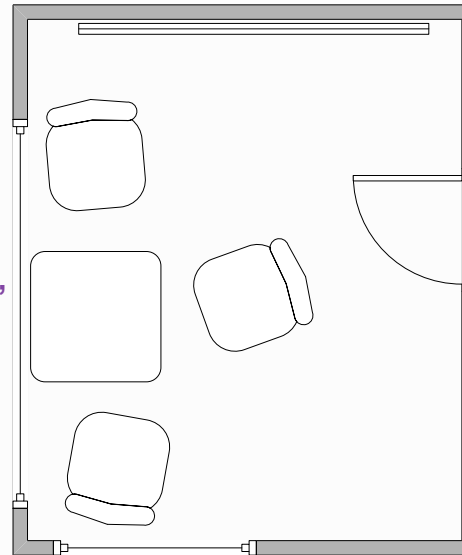
- USB, HDMI, Bluetooth, ...
- HTML, XML, ...
- ISO 9000, ISO/IEC 15504, ...
- IEEE
- etc...

Patterns: The Hillside Group - A group dedicated to design patterns. Home of the patterns library.

Patrones de Software (origen)

- Christopher Alexander (1977, 1979) - Arquitecto
 - “Cada patrón describe un problema que ocurre repetidas veces en nuestro entorno, y luego describe el núcleo de la solución para ese problema, de forma que usted pueda utilizar esa solución millones de veces sin usarla del mismo modo dos veces”
 - Propuesta de patrones extraídos a partir de estructuras de edificios y ciudades de diversas culturas, con el fin de ayudar a las personas a construir sus comunidades con la mejor calidad de vida posible
 - Ejemplo de un patrón:

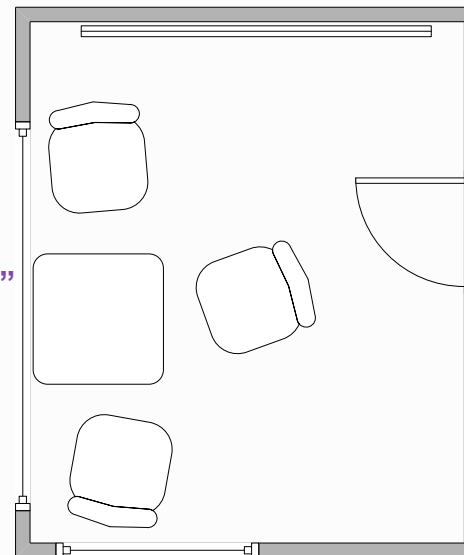
Window place
("un lugar en la ventana")



*Christopher Alexander et al, *A Pattern Language-Towns. Buildings. Construction*, 1977

Patrones de Software (origen)

Window place
("un lugar en la ventana")



*Christopher Alexander et al, *A Pattern Language- Towns. Buildings. Construction*, 1977

- A todo el mundo le encantan los asientos en la ventana, las ventanas salientes y las grandes ventanas con alféizares bajos y sillas cómodas arrastradas hacia ellas ... Una habitación sin un lugar como este rara vez permite que uno se sienta cómodo
- Si la habitación no contiene una ventana que sea un "lugar", una persona se verá impulsada a:
 - 1. Querer sentarse y estar cómodo
 - 2. Ser atraído por la luz
- Si los lugares cómodos están lejos de las ventanas, no hay forma de superar este conflicto...
- Por lo tanto: en todas las habitaciones donde pase algún tiempo durante el día, haga al menos un "lugar en la ventana"

Patrones de Software (origen)

Beck y Cunningham: 1987

pequeño lenguaje de patrones para guiar a programadores inexpertos en Smalltalk

Peter Coad: 1992

patrones de análisis descubiertos en el modelado de sistemas de información

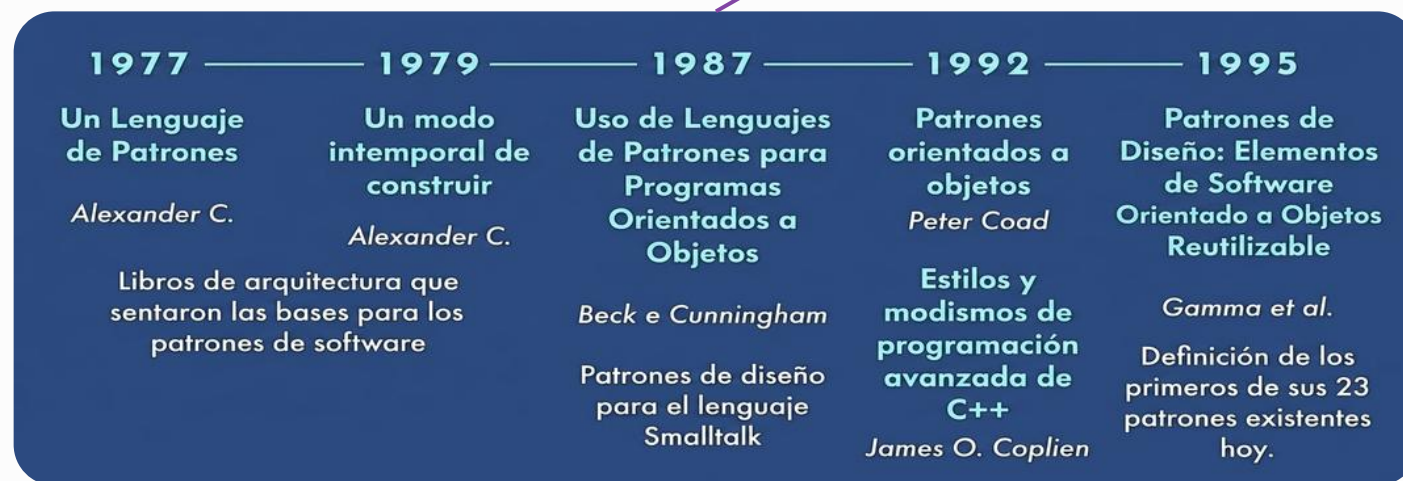
James Coplien – 1992

catálogo de varios estilos (“idioms”), con el fin de estandarizar la escritura de código en C+

Gamma et al – 1995

patrones de diseño derivados de la experiencia práctica con el desarrollo de software orientado a objetos

En software:



¿Por qué Patrones?

- Los desarrolladores acumulan soluciones para los problemas que resuelven con frecuencia
- Estas soluciones son difíciles de elaborar y pueden aumentar la productividad, calidad y uniformidad del software
- ¿Cómo documentar estas soluciones para que otros desarrolladores, menos experimentados, puedan utilizarlas?
- **Patrones de Software:** Describen soluciones para problemas que ocurren con frecuencia en el desarrollo de software (*Gamma 95*)

Un buen patrón:

- resuelve un problema
- es un concepto aprobado (no solo teoría)
- la solución no es obvia
- describe una relación (estructuras y mecanismos)
- tiene un componente humano significativo

Vantagens de los Patrones

- **Aumento de produtividade:** la aplicación de soluciones listas reduce el tiempo dedicado a pensar en la solución, diseñarla y luego aplicarla.
- **Uniformidad en la estructura del software:** entender un software es más fácil cuando se han utilizado patrones, ya que se pueden identificar los patrones utilizados a lo largo del software (ya sea en el diseño o en la implementación).
- **Aplicación inmediata por otros desarrolladores:** conociendo el patrón (su propósito, su estructura, el contexto en el que debe aplicarse, sus consecuencias, etc.) se puede aplicar inmediatamente incluso sin haber sido el proponente del patrón.
- **Reducción de la complejidad:** los patrones pasan a ser vistos como bloques constructivos, donde la complejidad inherente a cada patrón se abstrae y se ve el todo con menor complejidad (principio de dividir para conquistar).

Categorías de Patrones

- Organizacionales
- Arquitectónicos
- De Análisis
- De Diseño
 - **Gamma et al**
 - Otros : p.ej. J2EE
- De programación (estilos)
- De usabilidad
- Educativos
- Etc.

Tema de este
módulo del MBA

EJEMPLO

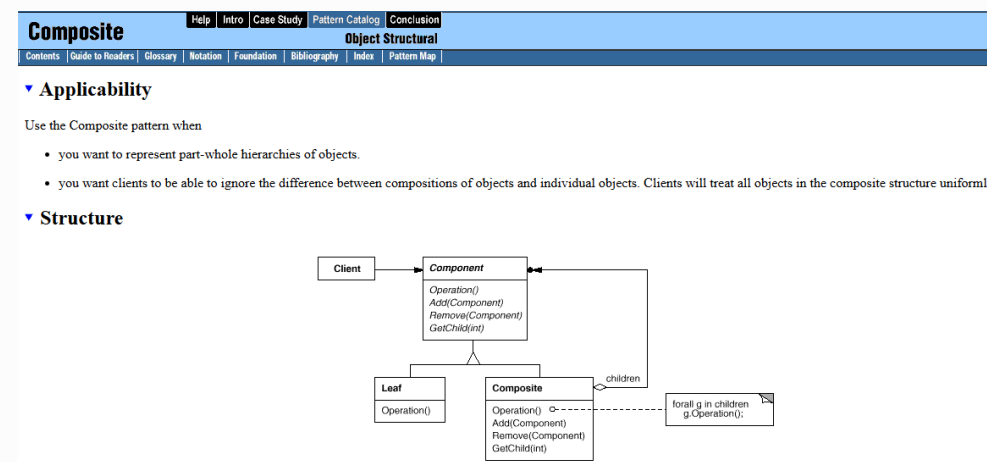
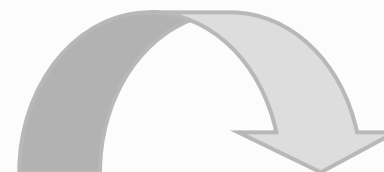


Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)

Elementos de un patrón



(Meszaros & Doble, 1998)

Elementos de un patrón



Encuesta 1 : Problema:
*“no tengo dinero para hacer
mi viaje de vacaciones”*



(Meszaros & Doble, 1998)

Elementos de un patrón



Encuesta 2 : Problema en un determinado contexto:

“no tengo dinero para hacer mi viaje de vacaciones”



(Meszaros & Doble, 1998)

Formato para la escritura de patrones de software

- Al escribir un patrón, se pueden combinar algunos elementos esenciales y otros elementos opcionales.
- En principio, no debe ceñirse a ningún formato de escritura y no hay obligatoriedad de usar todos los elementos en todos los patrones.
- A medida que el patrón se va reutilizando, se puede cambiar su formato para facilitar su comprensión y reutilización
- Existen algunas sugerencias de plantillas ya testeadas y aprobadas : **Gamma et al. (1995), Coplien (1996) y Buschmann et al. (1996)**
 - Obs: existen otros formatos

Formato para la escritura de patrones de software

Formato
utilizado en este
módulo del MBA

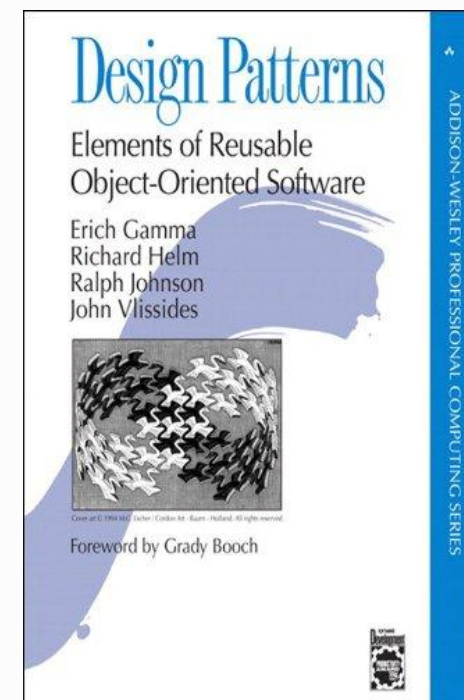
Gamma	Coplien	Buschmann
Nombre	Nombre	Nombre
Clasificación	Contexto	También Conocido Como
Intención	PROBLEMA	EJEMPLO
También Conocido Como	Fuerzas	Contexto
Motivación	Solución	PROBLEMA
Aplicabilidad	Esquema	Solución
Estructura	Contexto	Estructura
Participantes	Resultante	Dinámica
Colaboraciones	Razonamiento	Implementación
Implementación		Ejemplo Resuelto
Código Ejemplo		Variantes
Consecuencias		Usos Conocidos
Usos Conocidos		Consecuencias
Patrones Relacionados		Ver También

PATRONES DE DISEÑO:

Describen soluciones a problemas que ocurren con frecuencia en el diseño de software orientado a objetos

Tipos de Patrones de Diseño GoF*

- Catálogo de Patrones de Diseño publicado en 1995 por Gamma, Helm, Johnson y Vlissides
- Dos criterios de clasificación:
 - Propósito - refleja lo que hace el patrón
 - De Creación: trata de la creación de objetos
 - Estructural: se encarga de la composición de clases y objetos
 - Comportamental: caracteriza el modo en que las clases y objetos interactúan y distribuyen responsabilidades
 - Ámbito en el que actúa el patrón:
 - Clase: trata de la relación entre clases y subclases (herencia - relación estática)
 - Objetos: maneja la manipulación de objetos (pueden ser modificados en tiempo de ejecución)



*** GoF: Gang of Four – apodo dado a los cuatro autores del libro**

Patrones de Diseño GoF



Encuesta 3: Experiencia
con patrones GoF



		Propósito		
		De creación	Estructurales	Conductual
Alcance	Clase	Método de fábrica	Adaptador	Intérprete Template Method
	Objeto	- Fábrica Abstracta - Constructor - Prototipo - Objeto Único (Singleton)	- Adaptador - Puente - Compuesto - Decorador - Fachada - Peso Ligero - Proxy	- Cadena de Responsabilidad - Comando - Iterador - Mediador - Memento - Observador - Estado - Estrategia - Visitante

Antes de empezar...

- ¿Entendemos cómo la GoF estructuró la escritura de los 23 patrones de diseño?
 - **Nombre y clasificación del patrón:** El nombre del patrón transmite la esencia del patrón de forma sucinta. Un buen nombre es vital, porque se convertirá en parte de su vocabulario de diseño. La clasificación del patrón refleja el esquema presentado en la diapositiva anterior
 - **Intención:** Una breve declaración que responde a las siguientes preguntas: ¿Qué hace el patrón de diseño? ¿Cuál es su justificación e intención? ¿Qué problema o cuestión de diseño en particular aborda?
 - **También conocido como:** Otros nombres conocidos para el patrón, si los hay.
 - **Motivación:** Un escenario que ilustra un problema de diseño y cómo las estructuras de clases y objetos en el patrón resuelven el problema. El escenario le ayudará a comprender la descripción más abstracta del patrón que sigue.

Antes de empezar...

- **Aplicabilidad:** ¿Cuáles son las situaciones en las que se puede aplicar el patrón de diseño? ¿Cuáles son los ejemplos de diseños deficientes que el patrón puede abordar? ¿Cómo se pueden reconocer estas situaciones?
- **Estructura:** En el caso del libro de la GoF, se adoptó una representación gráfica de las clases en el patrón utilizando una notación basada en la Técnica de Modelado de *Objetos (Object Modeling Technique)*, una precursora de *UML (Unified Modeling Language)*. También se utilizan diagramas de interacción [JCJO92, Boo94] para ilustrar secuencias de solicitudes y colaboraciones entre objetos.
- **Participantes:** Las clases y/o objetos que participan en el patrón de diseño y sus responsabilidades.
- **Colaboraciones:** Cómo colaboran los participantes para ejecutar sus responsabilidades.

Antes de empezar...

- **Consecuencias:** ¿Cómo apoya el patrón sus objetivos? ¿Cuáles son las compensaciones y los resultados del uso del patrón? ¿Qué aspecto de la estructura del sistema le permite variar de forma independiente?
- **Implementación:** ¿Qué trampas, consejos o técnicas debe tener en cuenta al implementar el patrón? ¿Existen problemas específicos del lenguaje?
- **Código de ejemplo:** Fragmentos de código que ilustran cómo puede implementar el patrón en un lenguaje OO.
- **Usos conocidos:** Ejemplos del patrón encontrados en sistemas reales. El libro incluye al menos dos ejemplos de dominios diferentes.
- **Patrones relacionados:** ¿Qué patrones de diseño están estrechamente relacionados con este? ¿Cuáles son las diferencias importantes? ¿Con qué otros patrones debe usarse este?

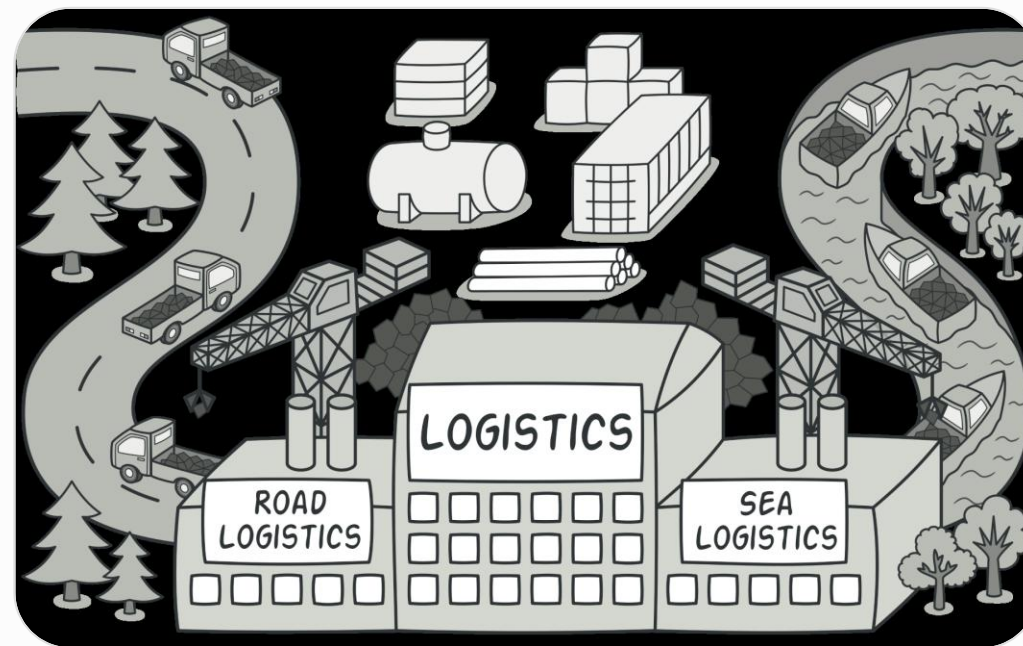
		Propósito		
		De creación	Estructurales	Conductual
Alcance	Clase	Método de fábrica	Adaptador	Intérprete Template Method
	Objeto	• Fábrica Abstracta • Constructor • Prototipo • Objeto Unitario (Singleton)	• Adaptador • Puente • Compuesto • Decorador • Fachada • Peso Ligero • Proxy	• Cadena de Responsabilidad • Comando • Iterador • Mediador • Memento • Observador • Estado • Estrategia • Visitante

PATRONES de la categoría “creacionales” o “de creación”

Encargados del proceso de creación de objetos, pudiendo aumentar la flexibilidad y reutilización de código

Patrón de Diseño Método de Fábrica (del inglés Factory Method)

- Imagine que ha lanzado una aplicación de gestión logística. En la primera versión, solo ofrece transporte por camiones, por lo que la mayor parte de su código reside dentro de la clase Camión
- Después de un tiempo, con el éxito de su aplicación, llegan solicitudes de empresas de transporte marítimo para incorporar también la logística marítima.
 - ¡Añadir la clase Barco requeriría cambios en todo el código!
 - El código estaría lleno de condicionales para tratar debidamente cada tipo de transporte, aunque hay mucho código que será útil independientemente del tipo de transporte.
- ¡¡¡Gran oportunidad para usar el patrón Método de Fábrica!!!



Shvets, Alexander, 2019

Patrón de Diseño Método de Fábrica (del inglés Factory Method)

- **Clasificación:** Patrón Estructural/Ámbito de clases
- **Intención:** Definir una interfaz para crear un objeto, pero dejar que las subclases decidan qué clase instanciar. El patrón permite que una clase posponga la instanciación a las subclases.
- **Motivación:** En varios tipos de aplicaciones, la elección de qué objeto se creará depende de decisiones tomadas en tiempo de ejecución, por lo que no hay forma de saber qué clase debe instanciarse. Esto puede causar mucho código duplicado para prever las diversas combinaciones de clases, además de estructuras case dispersas por el código.

Patrón de Diseño Método de Fábrica

- **Aplicabilidad**
- Use el patrón Método de fábrica cuando:
 - una clase no puede anticipar la clase de objetos que debe crear.
 - una clase quiere que sus subclasses especifiquen los objetos creados por ellas.
- **Estructura:**

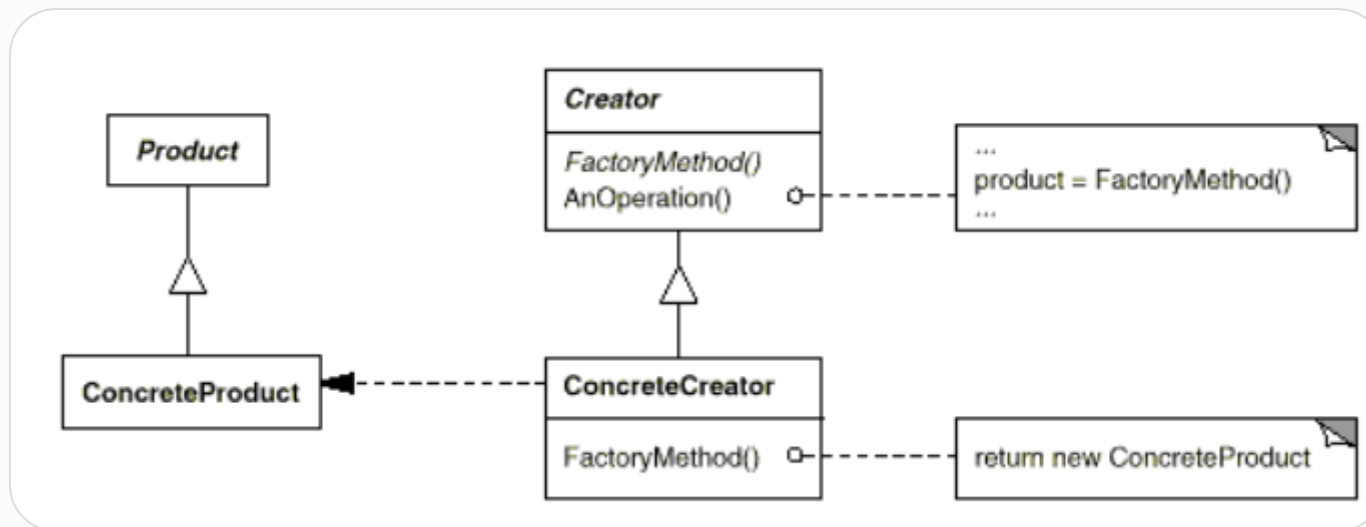


Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)

Patrón de Diseño Método de Fábrica

- **Participantes:**

- Declara el método de fábrica, que devuelve un objeto del tipo Product. También puede definir una implementación predeterminada que devuelve un objeto ConcreteProduct.
 - Puede llamar al método de fábrica para crear un objeto Product

- Define la interfaz de los objetos que el método de fábrica crea

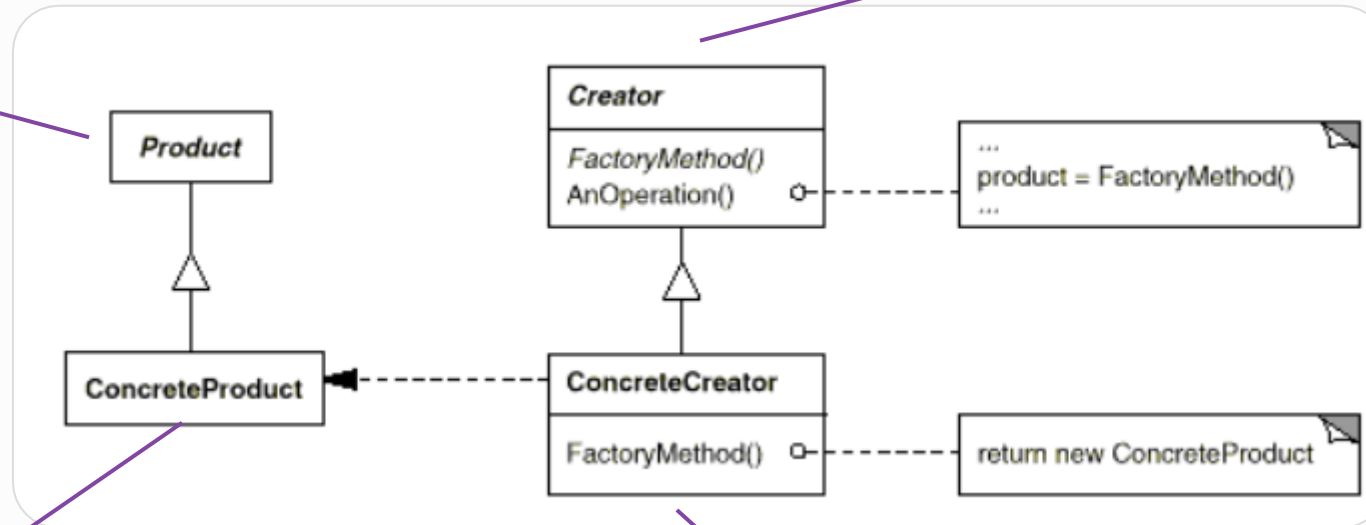
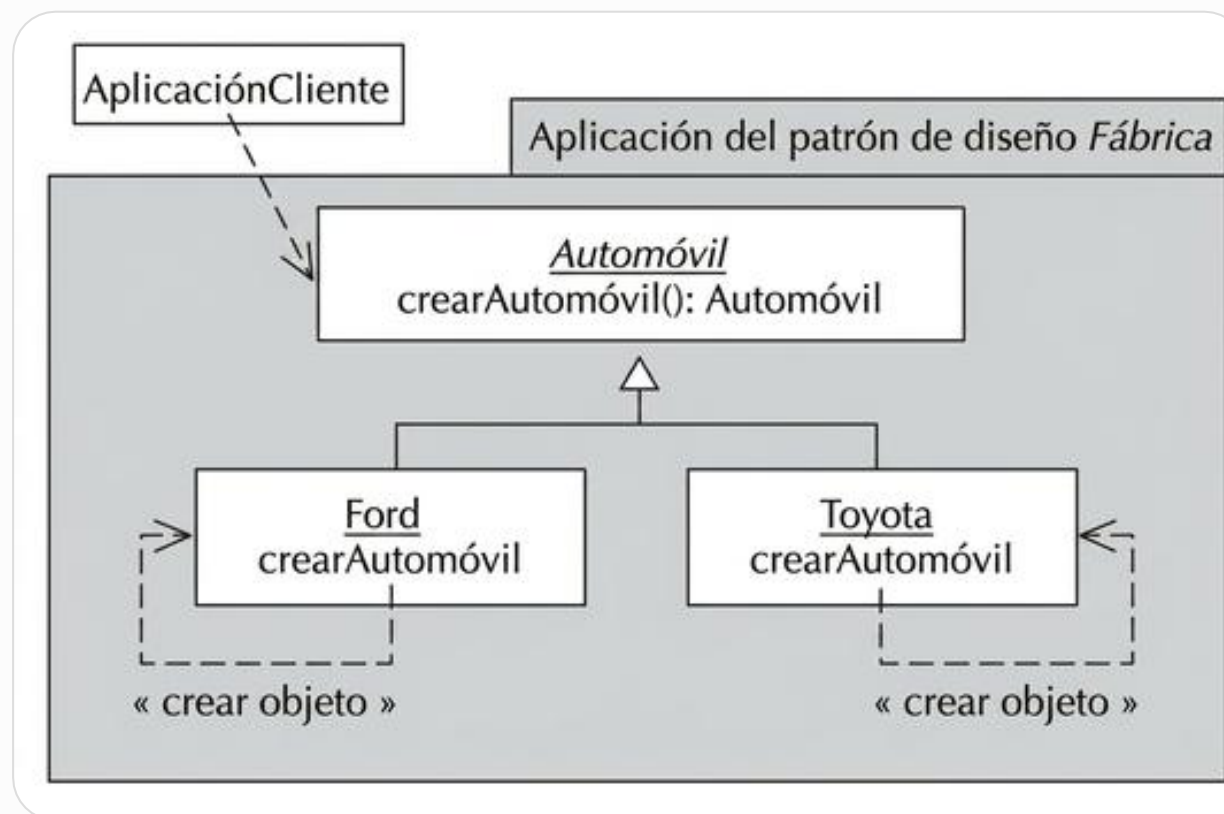


Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)

- Implementa la interfaz de Product

- Sobrescribe el método de fábrica para devolver una instancia de ConcreteProduct

Ejemplo de código del Patrón de Diseño Método de Fábrica



Braude, Eric, 2005

Ejemplo de código del Patrón de Diseño Método de Fábrica

```
abstract class Automovel // esta es la clase Product
{
    int añoFab;
    int getAñoFab()
    {
        return this.añoFab;
    }
    . . . // Otros atributos de automóvil entran aquí
    public double calcularSeguro(int añoFab)
    {
        return 0.00; // comportamiento default de la clase
    }
}

class Ford extends Automovil // esta es la clase ConcreteProduct
{
    . . .
    double calcularSeguro()
    {
        return this.añoFab + this.añoFab*3.123/2.12; // reglas específicas de Ford
    }
    . . .
}
class Toyota extends Automóvil // esta es otra clase ConcreteProduct
{
    . . .
    double calcularSeguro()
    {
        return this.añoFab_2 - this.añoFab_4.321/2.35; // reglas específicas de Toyota
    }
    . . .
}
```

Ejemplo de código del Patrón de Diseño Método de Fábrica

```
abstract class Corretora // esta es la clase Creator
{
public Automóvil criarAutomóvil() // método de fábrica
{
}
void renovarSeguro() // anOperation
{
Automóvil aut = criarAutomóvil();
int año = aut.obterAñoFab();
double v = aut.calcularSeguro();
System.out.println( "¡¡Seguro renovado!!" );
}
}

class CriadorFord extends Criador // esta es la clase ConcreteCreator
{
Ford criarAutomóvil()
{
return new Ford();
}
}
class CriadorToyota extends Criador // otra clase ConcreteCreator
{
Toyota criarAutomóvil()
{
return new Toyota();
}
}
```

Ejemplo de código del Patrón de Diseño Método de Fábrica

```
public static void main( String[] args )
{
Automóvil miAuto = obtenerTipoAutomóvil();
Correctora cor = new Correctora;
cor.renovarSeguro(miAuto); }

private static Automóvil obterTipoAutómovil()
{
System.out.println( "Por favor, elija 1 para Ford o 2 para Toyota);
try // seleccione la entrada del usuario
{ BufferedReader bufReader =
new BufferedReader( new InputStreamReader( System.in ) );
tipoDesejado = bufReader.readLine();
}
catch( IOException e )
{ System.out.println( e );
}
// Devuelve la instancia de 'Automóvil' de acuerdo con la entrada del
usuario
// este es el fragmento que necesita ser modificado si se quieren dejar
disponibles más tipos de automóvil
if( tipoDesejado == 1)
autoSeleccionado = CriadorFord.criarAutomóvil();
else
autoSeleccionado = CriadorToyota.criarAutomóvil();
return autoSeleccionado;
}
```

Consecuencias del uso del Método de Fábrica

- Evita un acoplamiento fuerte entre el creador y los productos concretos
- Facilita la inclusión de nuevos tipos de productos concretos, sin romper el código
- **Punto negativo:**
 - El código se vuelve más complejo ya que es necesario incluir nuevas clases para implementar el patrón

Patrones relacionados con el Método de Fábrica

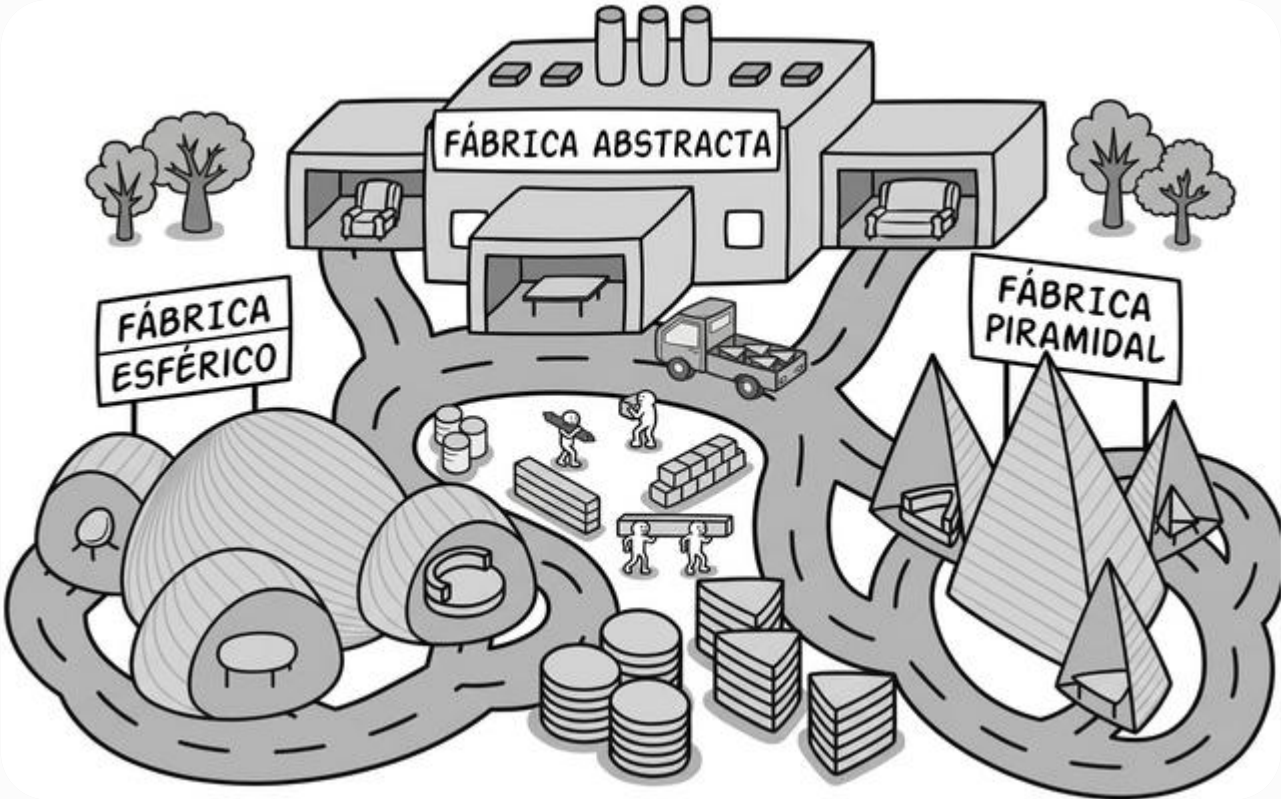
- Abstract Factory (¡lo veremos a continuación!)
- Template Method
- Prototype



Encuesta 4 : Uso del método de fábrica



Patrón de Diseño Fábrica abstracta (del inglés Abstract Factory)



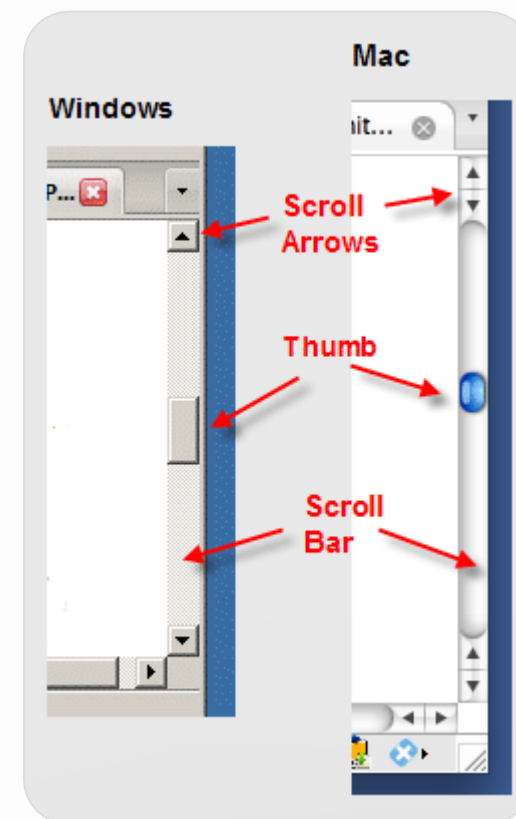
Shvets, Alexander, 2019

¿Y si necesito fabricar familias de objetos relacionados?

	Silla	Sofá	Mesa de centro
Art déco			
Victoriano			
Moderno			

Patrón de Diseño Fábrica abstracta

- **Clasificación:** Patrón Estructural/Ámbito de objetos
- **Intención:** Proporcionar una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
- **Motivación:** En muchas aplicaciones, es necesario crear un conjunto de objetos relacionados, pero que pueden tener una implementación diferente en cada plataforma.
 - Por ejemplo, en un sistema de interfaz gráfica puede haber “widgets” como botones, ventanas, barras de desplazamiento para Windows o para MacOS. Si es necesario realizar la portabilidad entre diferentes S.O.s, el código estará acoplado a ciertas clases y necesita ser modificado por completo
 - Elegido un estilo, todos los elementos deben ser siempre utilizados en conjunto
 - Pero hay comportamientos comunes entre los elementos de las diferentes familias, por ejemplo: un botón siempre tiene los mismos tipos de acción: clic (izquierdo y derecho), desplazamiento, etc., ya sea Windows o MacOS.



Patrón de Diseño Fábrica abstracta

- **Aplicabilidad** - Use el patrón Fábrica abstracta cuando:
 - un sistema debe ser independiente de cómo se crean, componen y representan sus productos.
 - un sistema debe ser configurado con una de varias familias de productos.
 - una familia de objetos de productos relacionados está diseñada para ser usada en conjunto, y necesita imponer esa restricción.
 - quiere proporcionar una biblioteca de clases de productos, y quiere revelar solo sus interfaces, no sus implementaciones.

- **Estructura:**

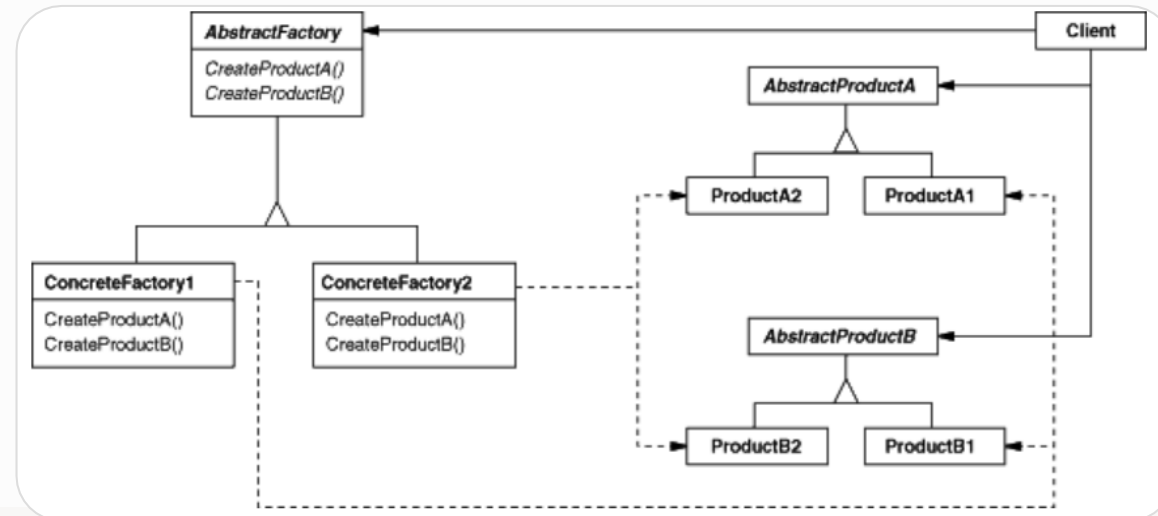


Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)

Patrón de Diseño Fábrica abstracta

- Participantes:

- Declara una interfaz para operaciones que crean los objetos abstractos

- Implementa las operaciones para crear los objetos concretos

- Declara una interfaz para un tipo de producto

Utiliza solo interfaces declaradas por AbstractFactory y AbstractProduct

- Define un objeto a ser creado por la fábrica concreta correspondiente
 - Implementa la interfaz AbstractProduct

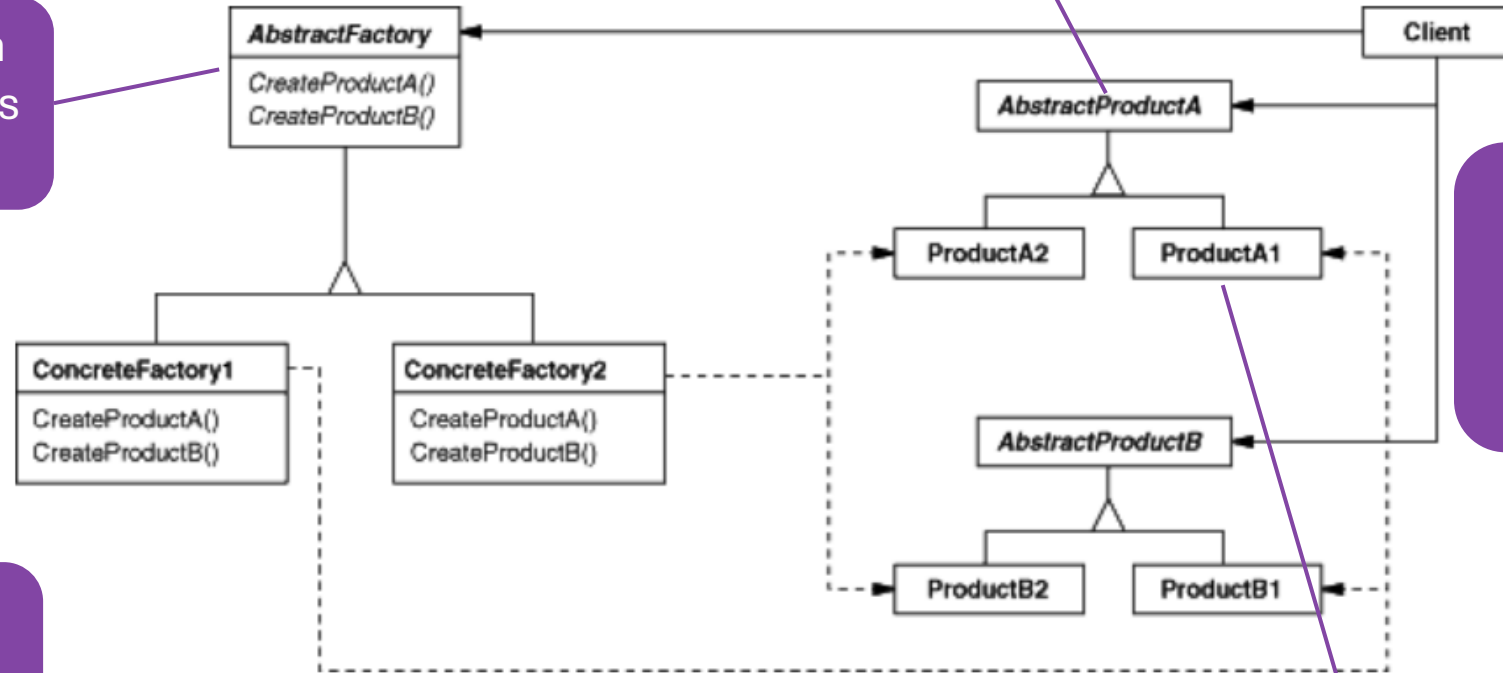


Imagen capturada en el libro
versión html publicado en CD
(Gamma et al 1995)

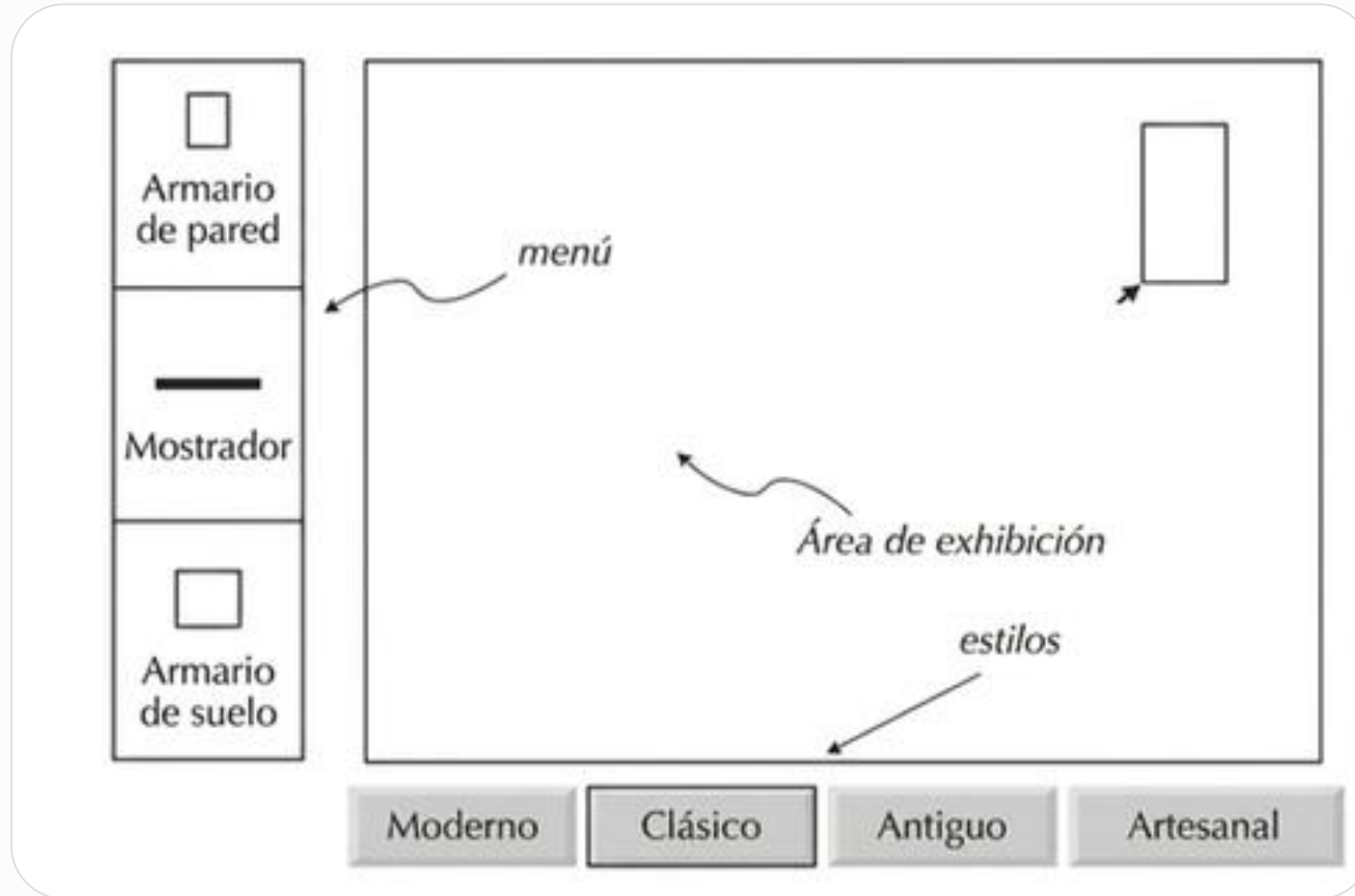
Patrón de Diseño Fábrica abstracta

Ejemplo del Patrón Fábrica abstracta

- Aplicación para construir armarios de cocina
- Los propietarios de viviendas siempre tienen la intención de modernizar sus cocinas, frecuentemente utilizando un software para visualizar las posibilidades.
- Visualizador De Cocinas: aplicación que permite al usuario crear el diseño de las partes de una cocina, sin comprometerse con un estilo.

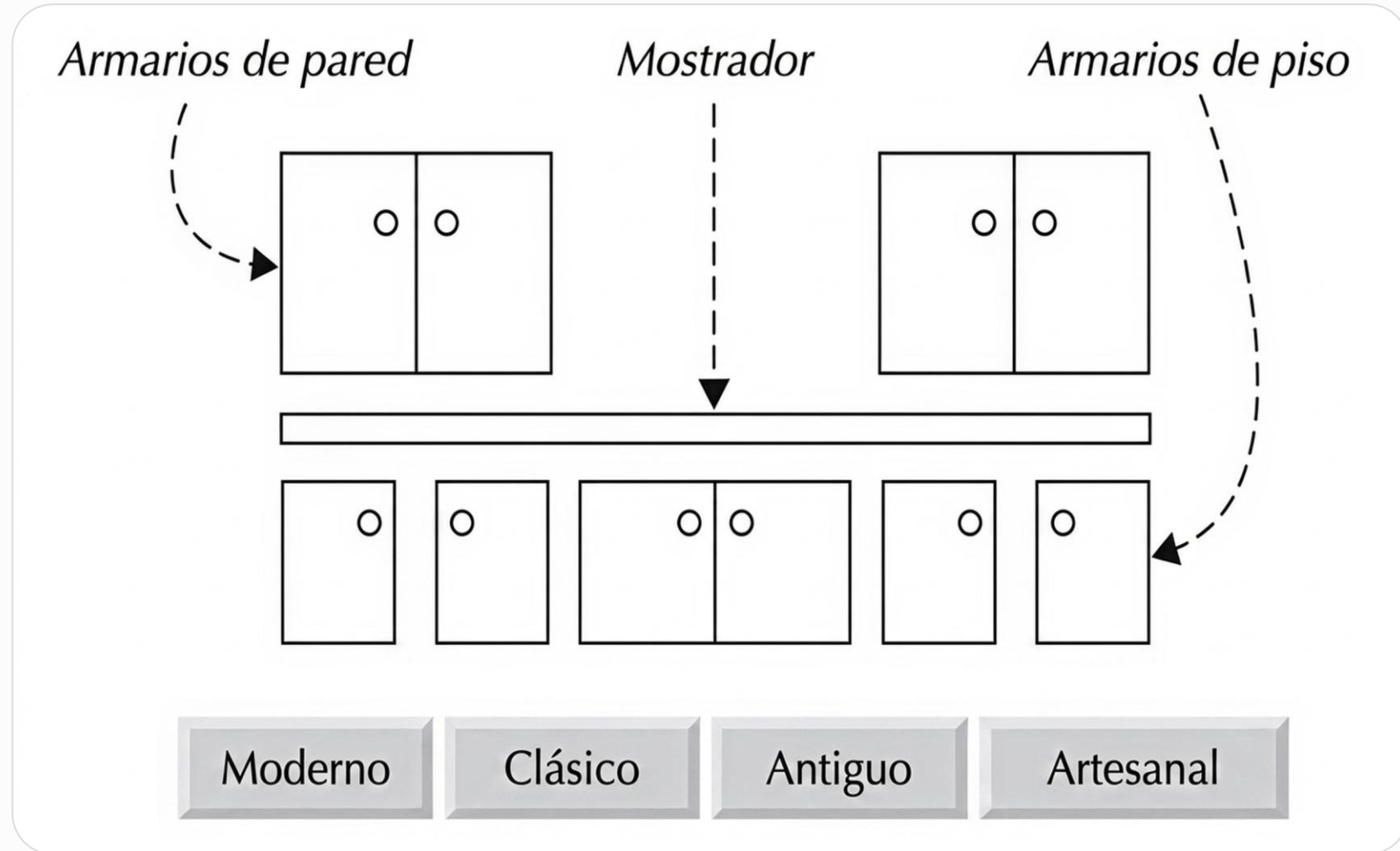
Patrón de Diseño Fábrica abstracta

Ejemplo del Patrón Fábrica abstracta



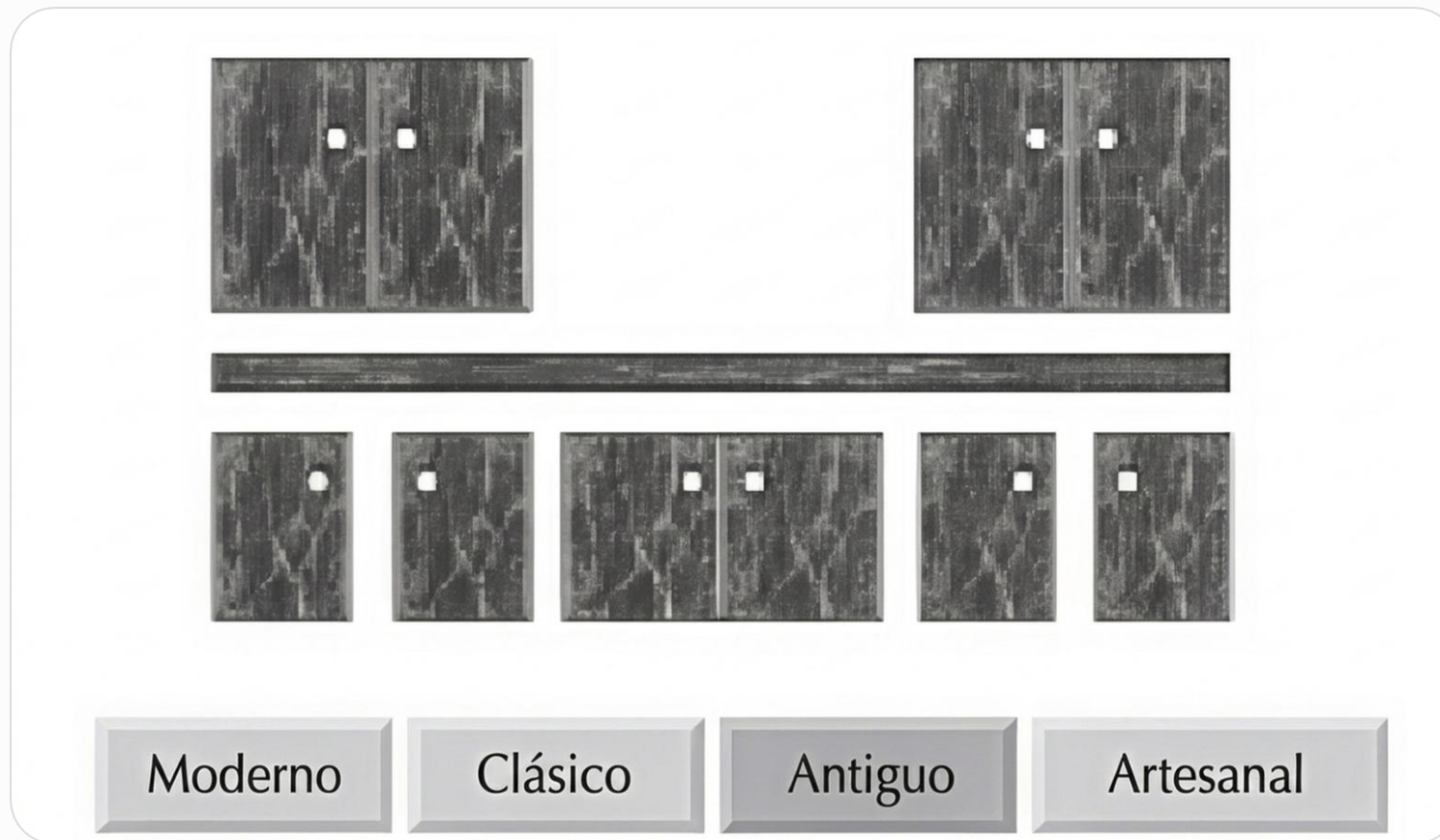
Patrón de Diseño Fábrica abstracta

Ejemplo del Patrón Fábrica abstracta



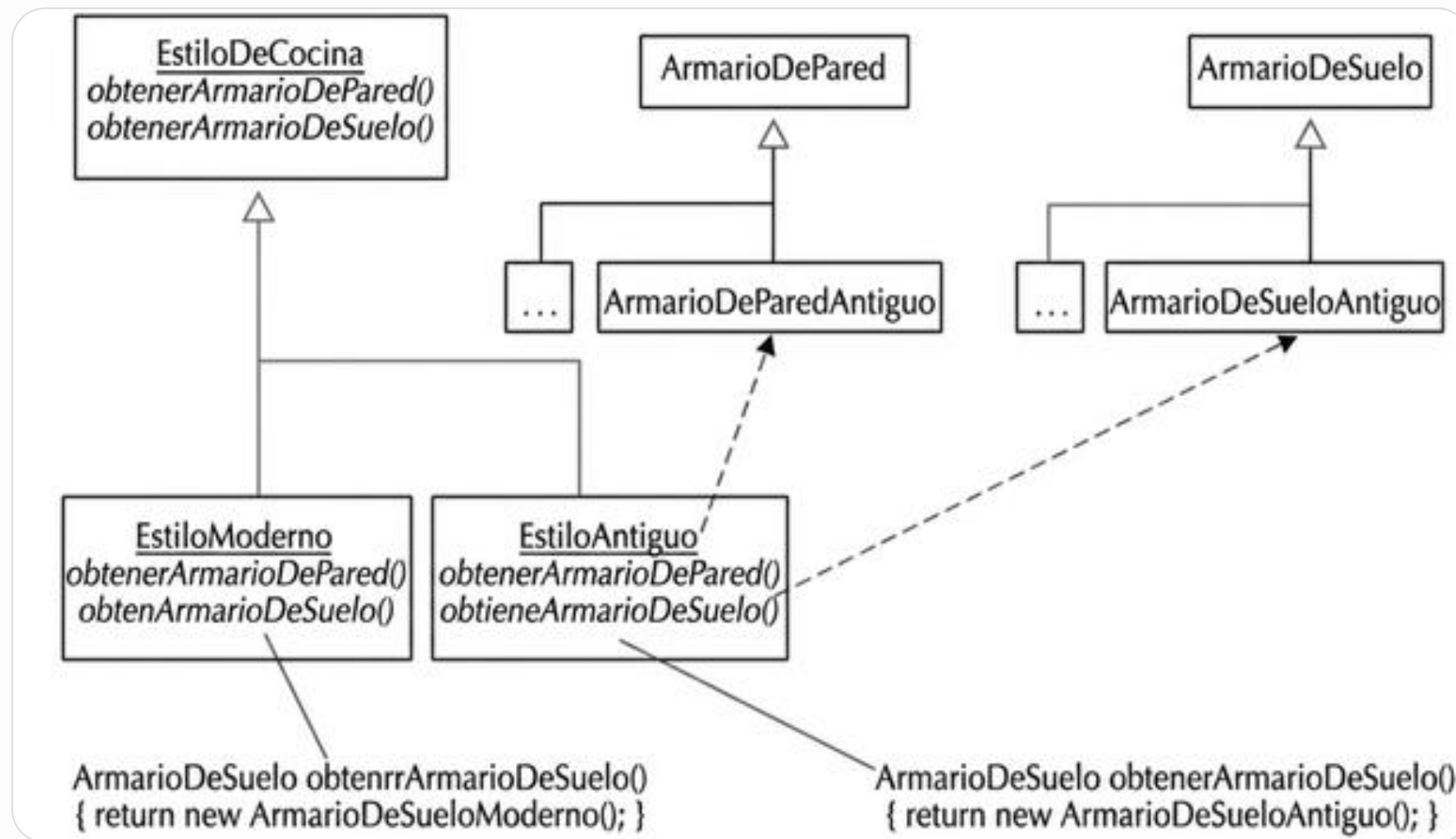
Patrón de Diseño Fábrica abstracta

Ejemplo del Patrón Fábrica abstracta



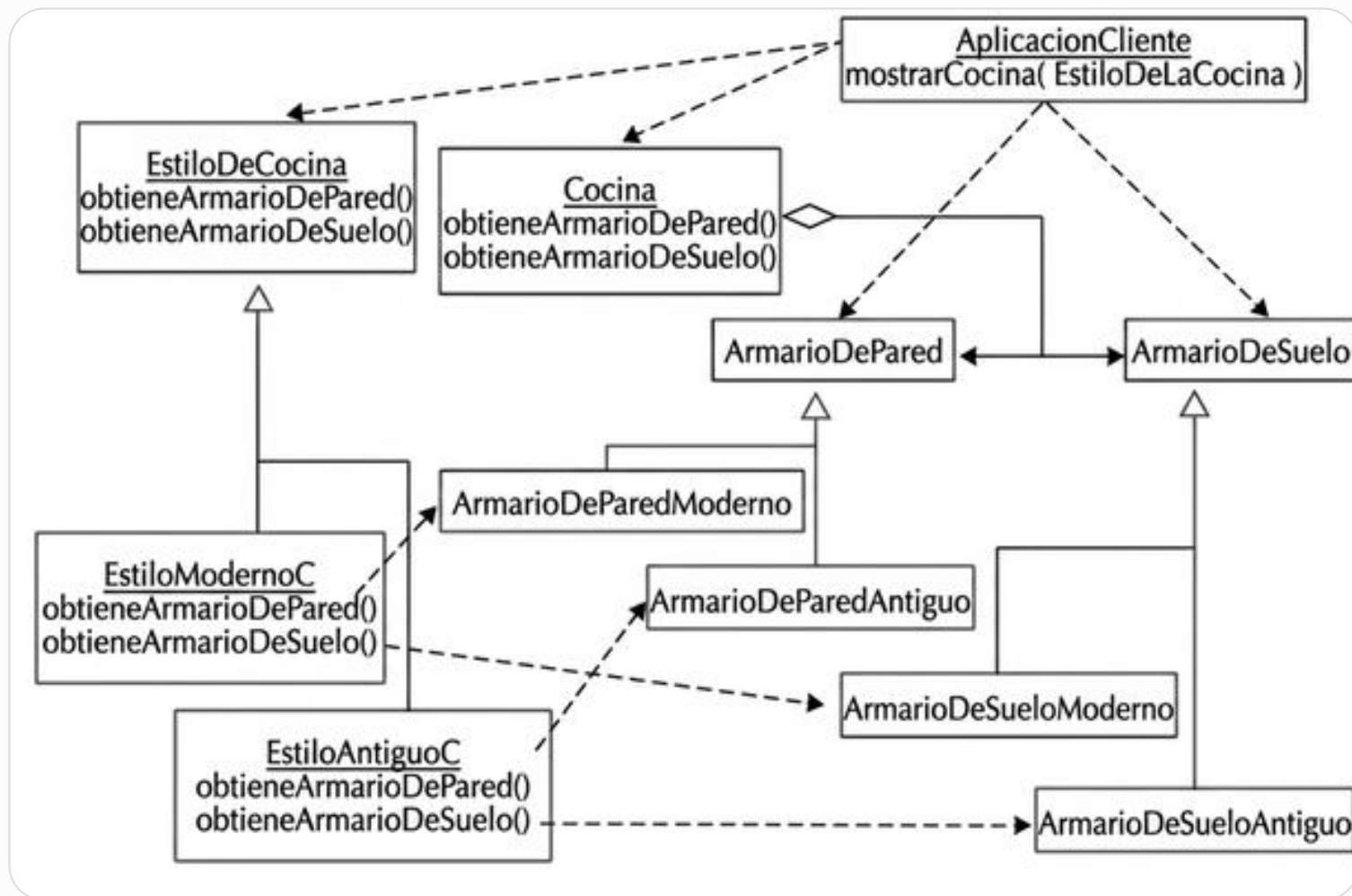
Patrón de Diseño Fábrica abstracta

Ejemplo del Patrón Fábrica abstracta



Patrón de Diseño Fábrica abstracta

Ejemplo del Patrón Fábrica abstracta



Patrón de Diseño Fábrica abstracta

Ejemplo del Patrón Fábrica abstracta

```
//Determina el estilo instanciando miEstilo
EstiloAntiguoC miEstilo = new EstiloAntiguoC;
// Crea los armarios de pared: Tipo determinado por la clase de
miEstilo
ArmarioDePared ArmarioDePared1 = miEstilo.obtenerArmarioDePared();
ArmarioDePared ArmarioDePared2 = miEstilo.obtenerArmarioDePared();
. . .
// Crea los armarios de suelo: Tipo determinado por la clase de
miEstilo
// Crea el objeto cocina (en el estilo requerido)
ArmarioDeSuelo armarioDeSuelo1 = miEstilo.obtenerArmarioDeSuelo();
ArmarioDeSuelo armarioDeSuelo2 = miEstilo.obtenerArmarioDeSuelo();
. . .
Cocina cocina = new Cocina();
Cocina.añadir( armarioDePared1, . . . );
Cocina.añadir( armarioDePared2, . . . );
. . .
Cocina.añadir( armarioDeSuelo1 . . . );
Cocina.añadir( armarioDeSuelo2 . . . );
. . .
```

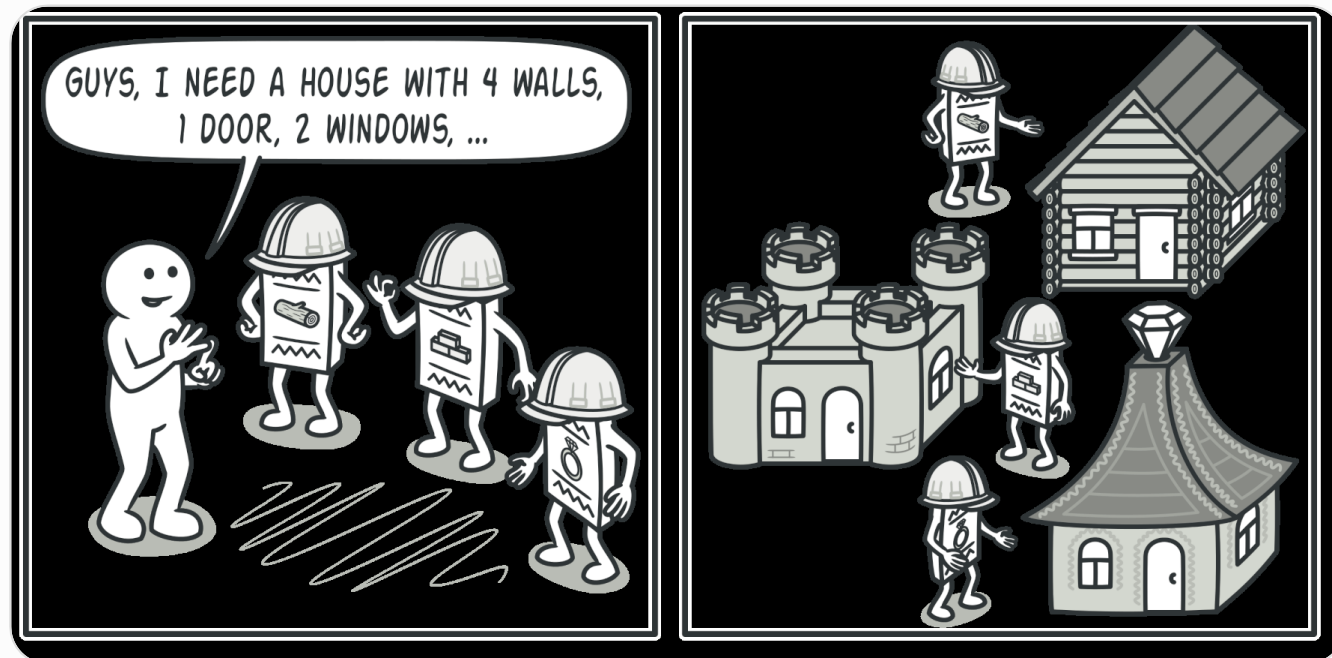
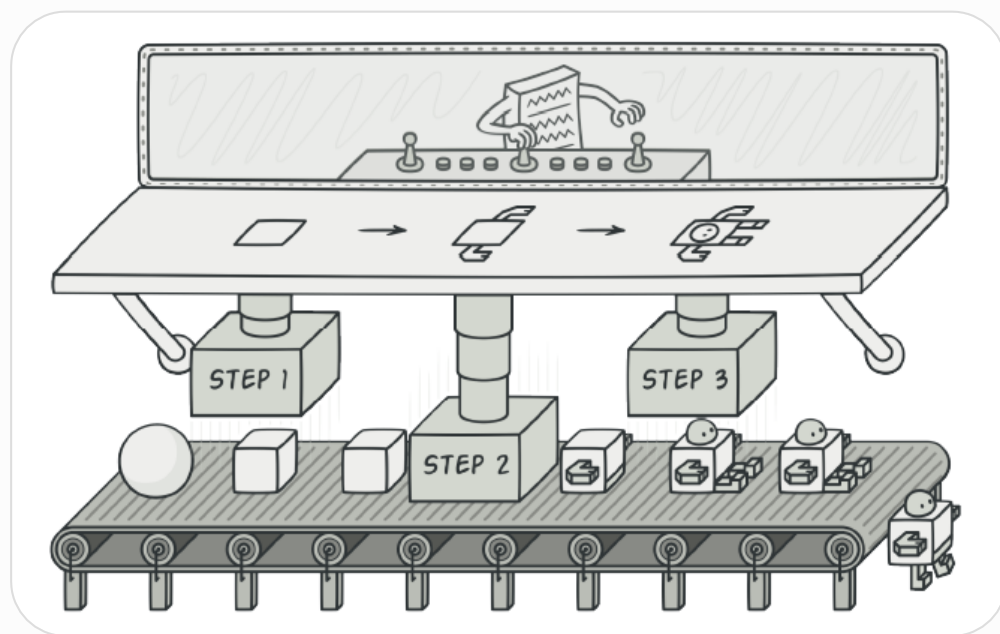
Patrón de Diseño Fábrica abstracta

Ejemplo del Patrón Fábrica abstracta

```
//Determina el estilo instanciando miEstilo
EstiloModernoC miEstilo = new EstiloModernoC;
// Crea los armarios de pared: Tipo determinado por la clase de
miEstilo
ArmarioDePared ArmarioDePared1 = miEstilo.obtenerArmarioDePared();
ArmarioDePared ArmarioDePared2 = miEstilo.obtenerArmarioDePared();
. . .
// Crea los armarios de suelo: Tipo determinado por la clase de
miEstilo
// Crea el objeto cocina (en el estilo requerido)
ArmarioDeSuelo armarioDeSuelo1 = miEstilo.obtenerArmarioDeSuelo();
ArmarioDeSuelo armarioDeSuelo2 = miEstilo.obtenerArmarioDeSuelo();
. . .
Cocina cocina = new Cocina();
Cocina.añadir( armarioDePared1, . . . );
Cocina.añadir( armarioDePared2, . . . );
. . .
Cocina.añadir( armarioDeSuelo1 . . . );
Cocina.añadir( armarioDeSuelo2 . . . );
. . .
```

Patrón de Diseño Constructor (del inglés Builder)

- **Clasificación:** Patrón de Creación/Ámbito de objetos
- **Intención:** Separar la construcción de un objeto complejo de su representación para que el mismo proceso de construcción pueda crear representaciones diferentes.



(Shvets, Alexander, 2018)

Patrón de Diseño Constructor

- Estructura:

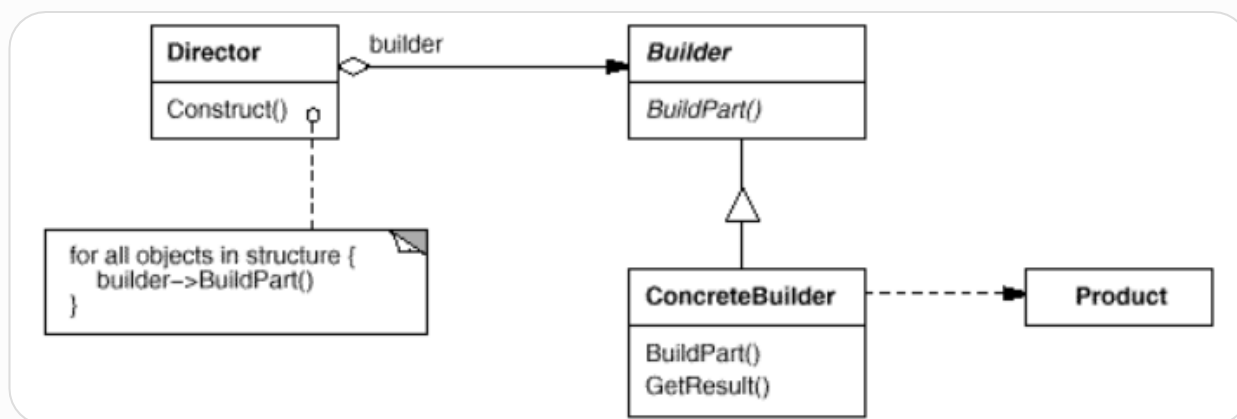
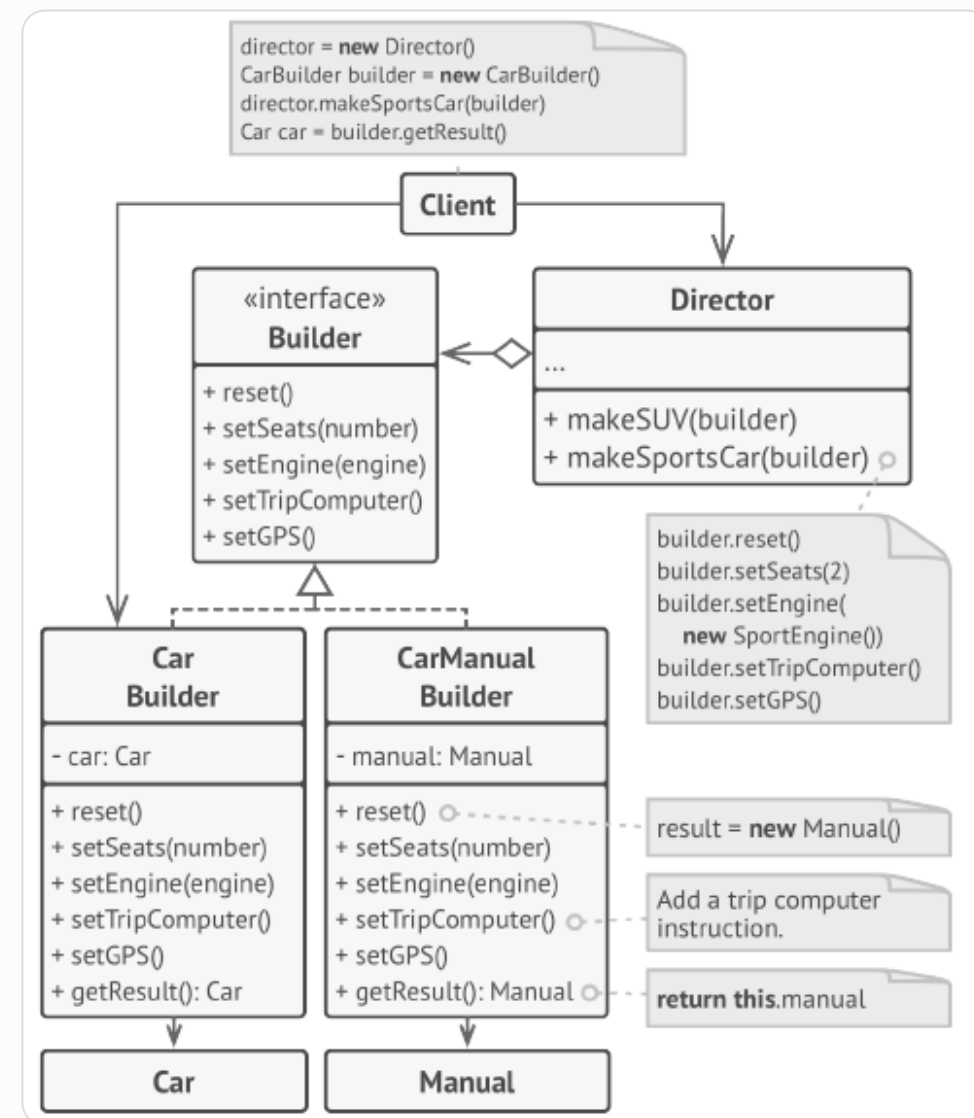


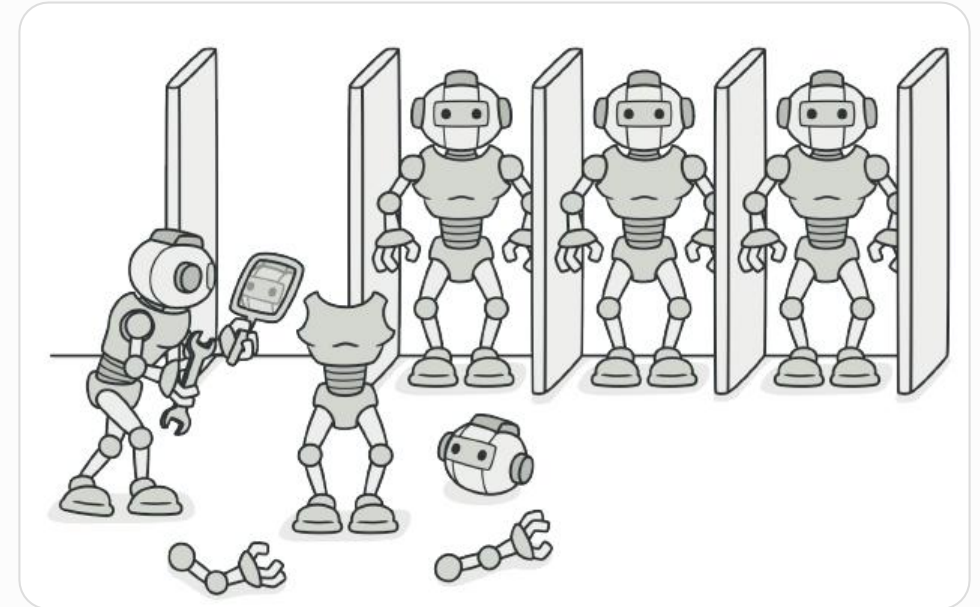
Imagen capturada en el libro versión html
publicado en CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Patrón de Diseño Prototipo (del inglés Prototype)

- **Clasificación:** Patrón de Creación/Alcance de objetos
- **Intención:** Especificar los tipos de objetos a crear utilizando una instancia prototípica y crear nuevos objetos copiando este prototipo, haciendo que el código no dependa de la clase de esos objetos
- **Aplicabilidad:** casos en los que los objetos se pasan a través de código de terceros mediante alguna interfaz. Las clases concretas de estos objetos son desconocidas, y no podría depender de ellas incluso si quisiera.



(Shvets, Alexander, 2018)

Patrón de Diseño Prototipo

- Estructura:

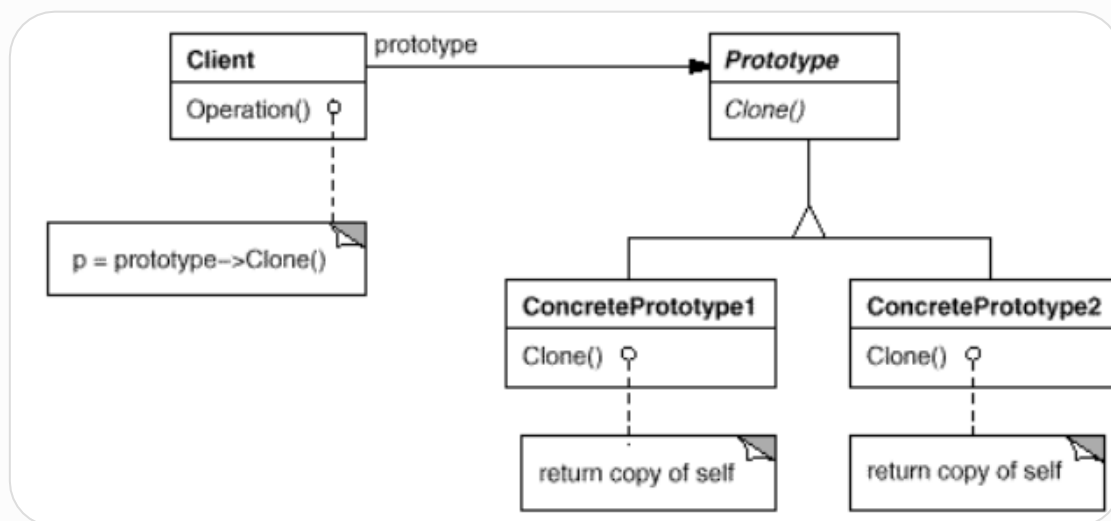
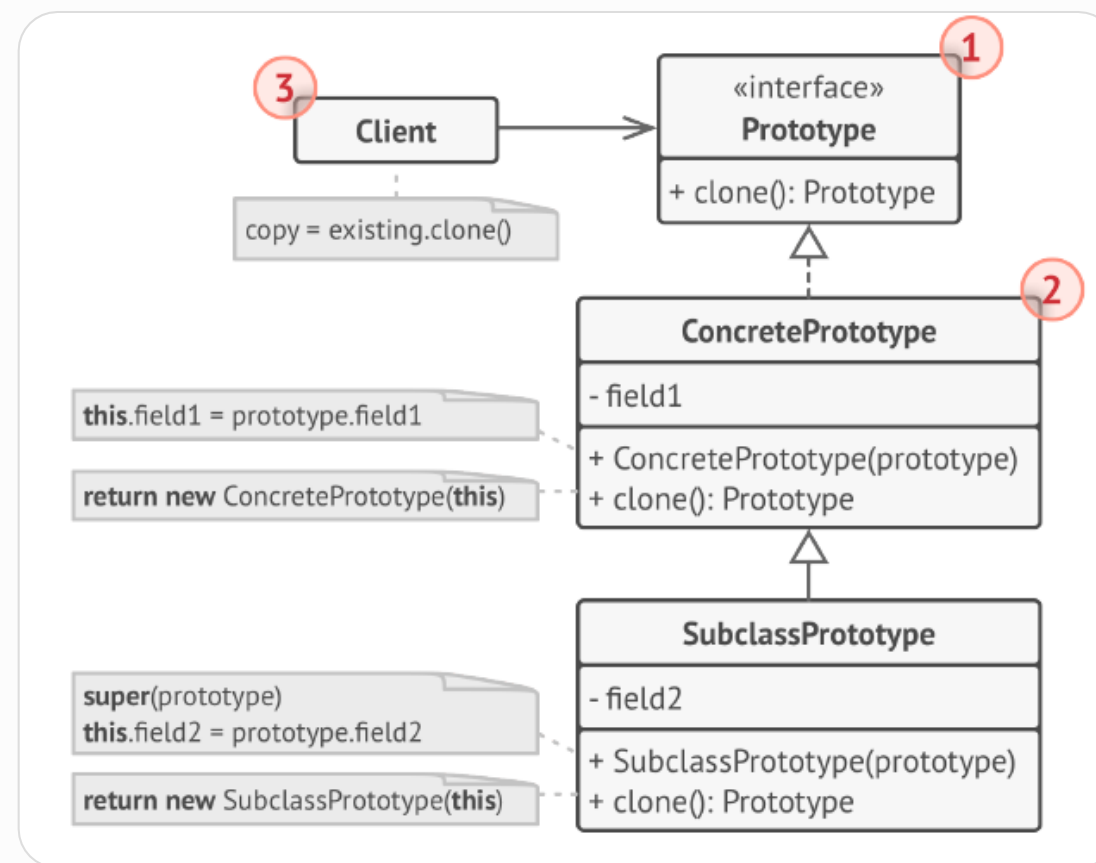


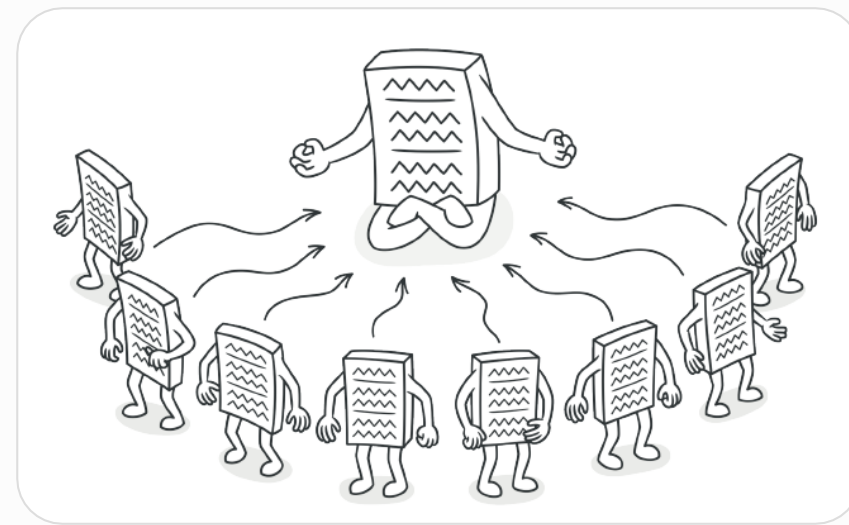
Imagen capturada en el libro versión html
publicado en CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Patrón de Diseño Objeto Unitario (del inglés Singleton)

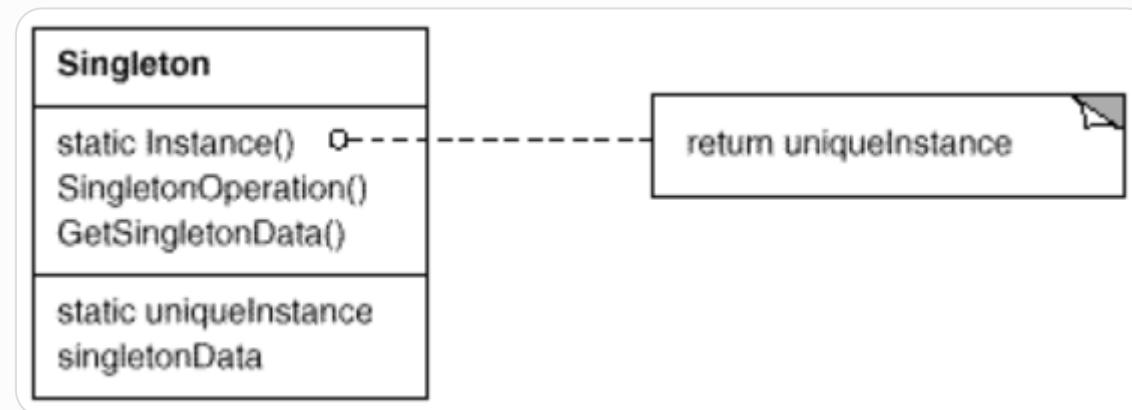
- **Clasificación:** Patrón de Creación/Alcance de objetos
- **Intención:** Garantizar que una clase tenga solo una instancia y proporcione un punto global de acceso a ella.
- **Motivación:** En algunas aplicaciones es importante garantizar que algunas clases tengan exactamente una instancia, por ejemplo, solo un sistema de archivos y un administrador de ventanas.
 - ¿Una variable global podría resolver el problema?



(Shvets, Alexander, 2018)

Patrón de Diseño Singleton

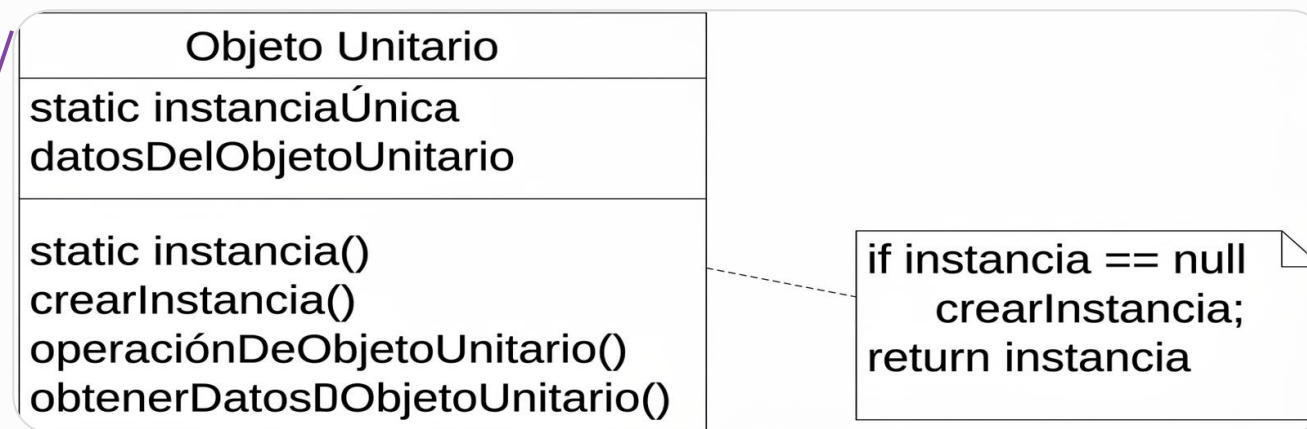
- **Aplicabilidad**
- Use el patrón Singleton cuando:
 - debe haber exactamente una instancia de una clase, y debe ser accesible a los clientes de manera fácil.
 - cuando la única instancia debe ser extensible por medio de herencia, y los clientes deben ser capaces de usar una instancia extendida sin modificar su código.
- **Estructura:**



Imágenes capturadas en el libro versión html publicado en CD (Gamma et al 1995)

Patrón de Diseño Singleton

- Participante:



(Braude, Eric, 2005)

- Define una operación Instancia() (en la clase) que permite a los clientes acceder a la instancia única.

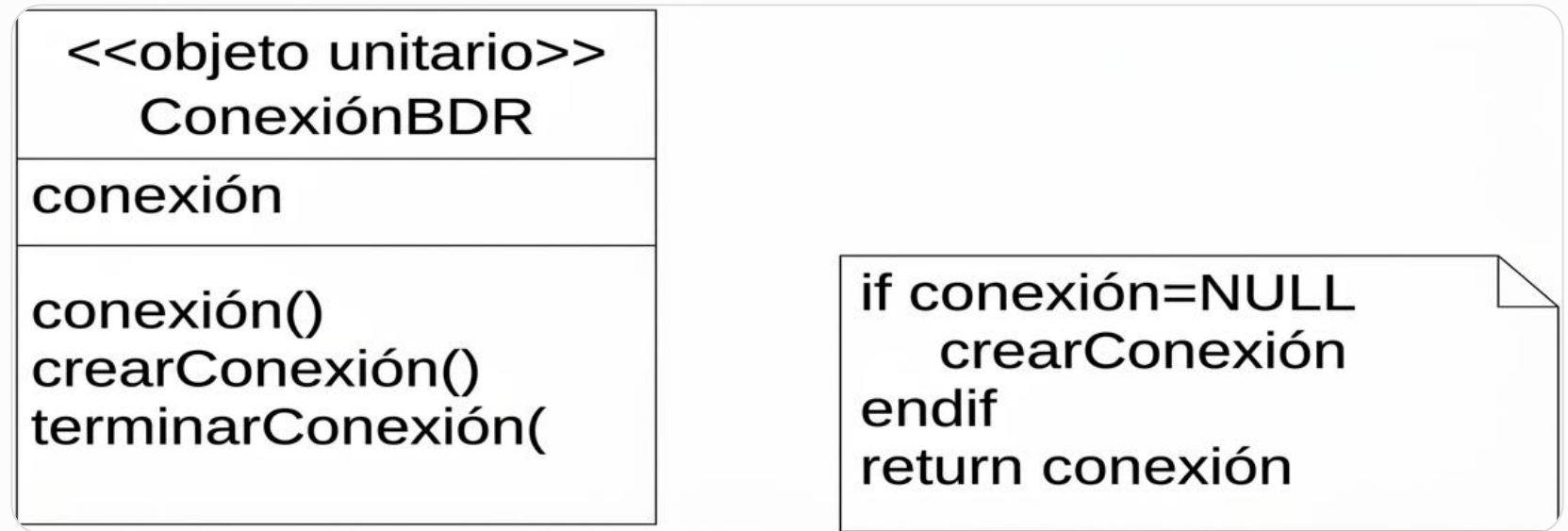
Patrón de Diseño Singleton

Ejemplo del Patrón Singleton

- Al iniciar una aplicación, es necesario establecer una conexión con la Base de datos, para tener un canal de comunicación abierto durante la ejecución de la aplicación.
- En general, esto se implementa mediante una clase Conexión (varias interfaces de BDRs existentes proporcionan una clase Connection especialmente para este fin).
- Esta conexión debe cerrarse posteriormente, al finalizar la aplicación.
- La conexión con el BDR debe ser preferentemente única, ya que si cada vez que se crea un objeto se crea una nueva conexión, existirán innumerables objetos en la memoria, innecesariamente.
- Así, se puede aplicar el patrón Singleton para garantizar que, para cada aplicación en ejecución, exista una única conexión válida con el BDR.

Patrón de Diseño Singleton

Ejemplo del Patrón Singleton



Ejemplo del Patrón Singleton

```
// final -> evita que se realice una herencia
public final class ConnectionPool {

    private static final String DATA_SOURCE_MYSQL = "java:comp/env/jdbc/passeLivre";
    private DataSource dataSource;           // pool de conexión con la base de datos
    private static ConnectionPool mySelf;    // referencia a una única instancia de esa clase

    // constructor privado
    private ConnectionPool( DataSource dataSource ) {

        this.dataSource = dataSource;
    }
    // synchronized para evitar que se cree más de una instancia en un sistema multihilo
    public static synchronized ConnectionPool getInstance() {

        try {
            // verifica si aún no se ha creado una única instancia
            if ( mySelf == null ) {

                // obtiene el contexto de la aplicación
                Context contexto = new InitialContext();
                // obtiene el pool de conexiones con la base de datos.
                DataSource dataSource = ( DataSource )contexto.lookup( DATA_SOURCE_MYSQL );
                // crea la única instancia de esa clase
                mySelf = new ConnectionPool( dataSource );
            }
        } catch( NamingException e ) {

            System.err.println( e.getMessage() );
        }

        return mySelf;
    }

    public Connection getConnection() throws SQLException {

        return dataSource.getConnection();
    }
}
```



Encuesta 5: Uso del Singleton



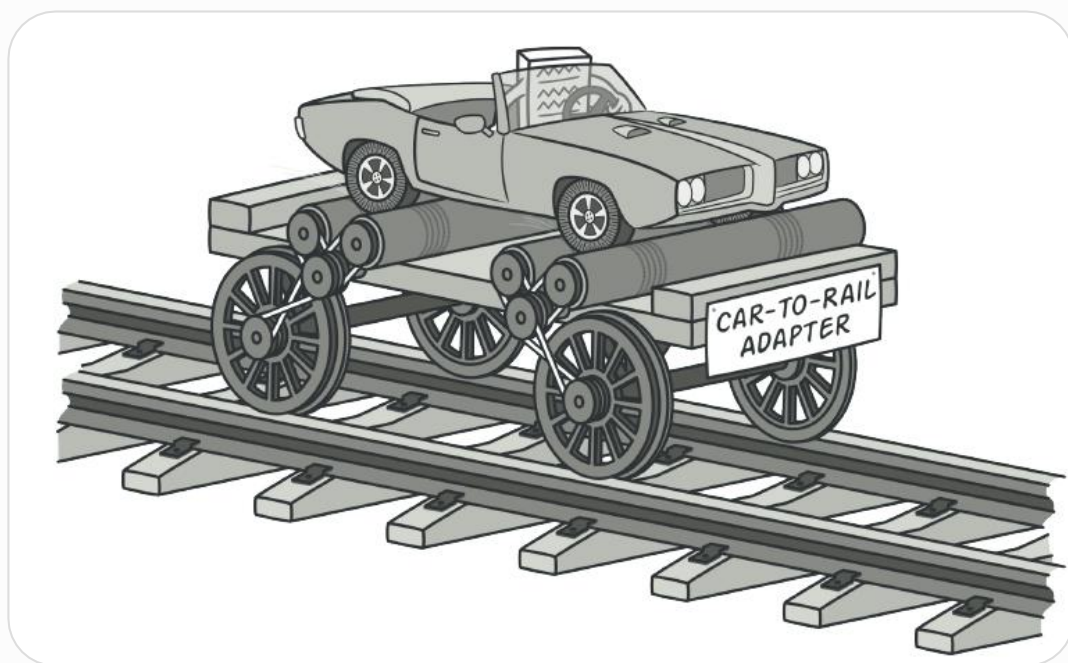
		Propósito		
		De creación	Estructurales	Conductual
Alcance	Clase	Método de fábrica	Adaptador	Intérprete Template Method
	Objeto	- Fábrica Abstracta - Constructor - Prototipo - Objeto Unitario (Singleton)	- Adaptador - Puente - Compuesto - Decorador - Fachada - Peso Ligero - Proxy	- Cadena de Responsabilidad - Comando - Iterador - Mediador - Memento - Observador - Estado - Estrategia - Visitante

PATRONES de la categoría “estructural”

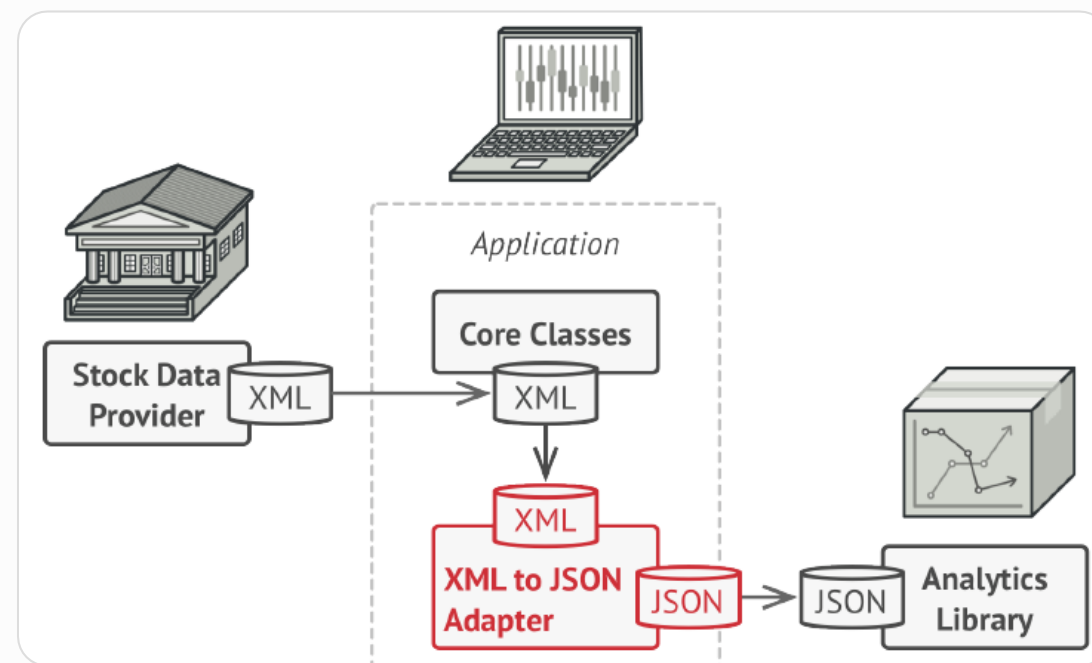
Se preocupan por la forma en que ocurren las composiciones de clases y objetos en estructuras mayores

Patrón de Diseño Adaptador (del inglés Adapter)

- **Clasificación:** Patrón estructural/Ámbito de clases/objetos
- **Intención:** Convertir la interfaz de una clase en otra interfaz que los clientes esperan. El Adaptador permite que clases con interfaces incompatibles puedan trabajar juntas

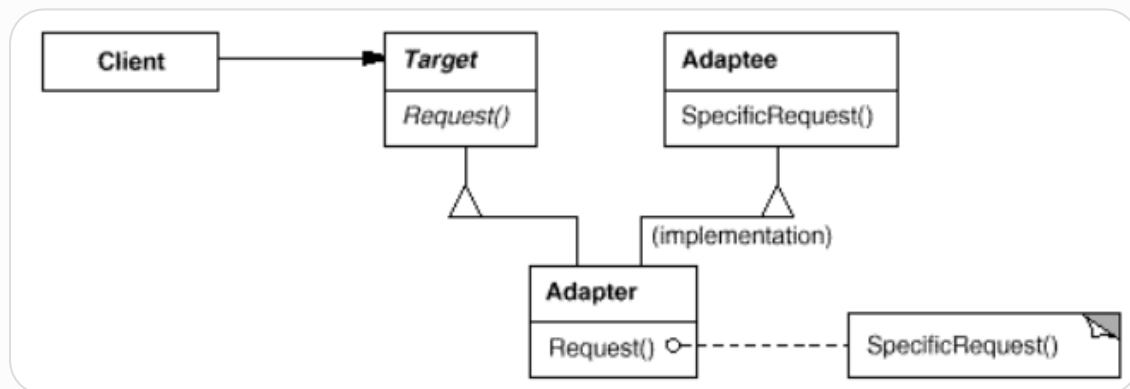


(Shvets, Alexander, 2018)

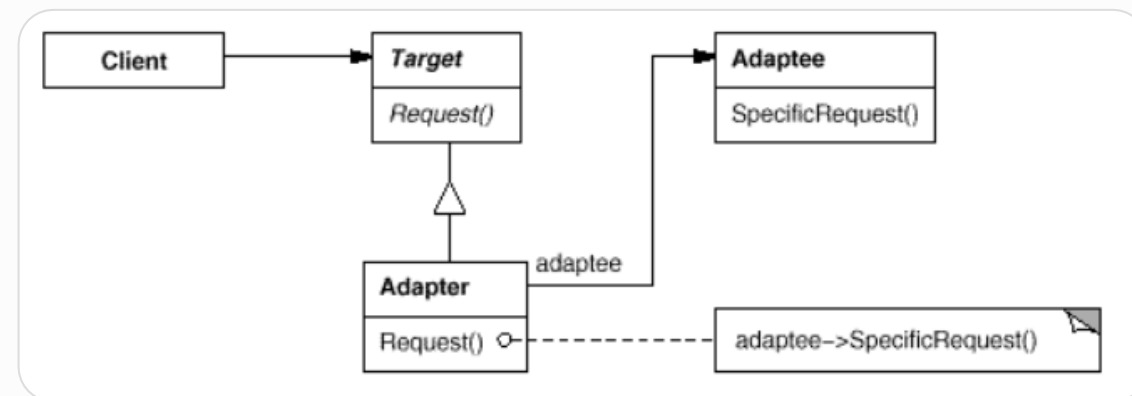


Patrón de Diseño Adaptador

- Estructura:



Adaptador en el ámbito de clase

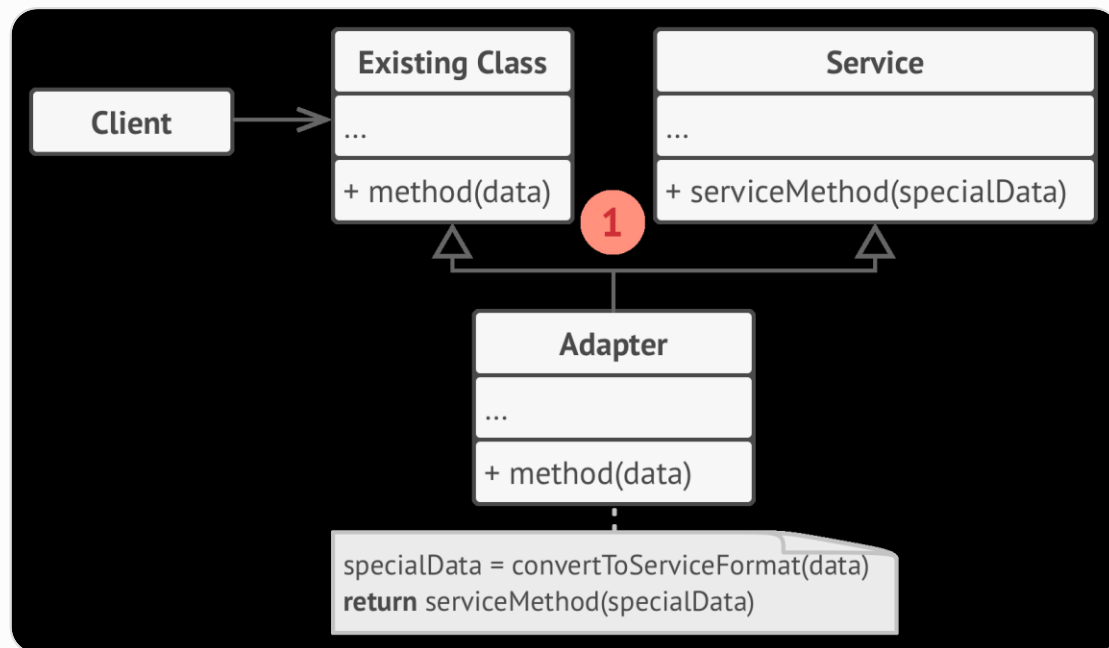


Adaptador en el ámbito de objeto

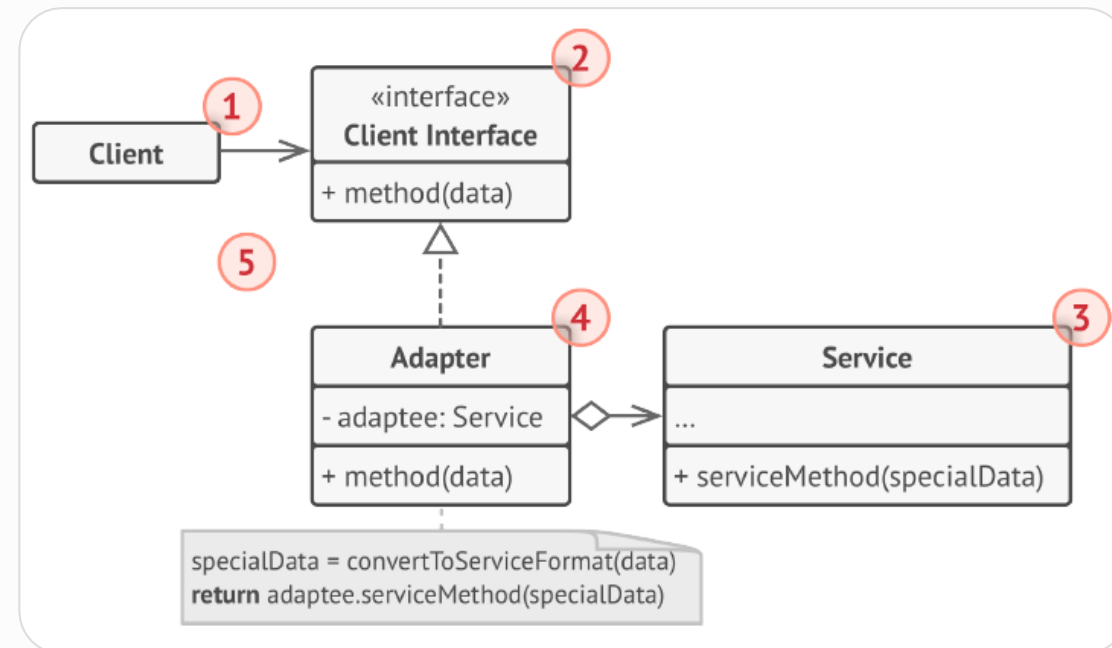
Imágenes capturadas en el libro versión html publicado en CD (Gamma et al 1995)

Patrón de Diseño Adaptador

- Estructura:



Adaptador en el ámbito de clase

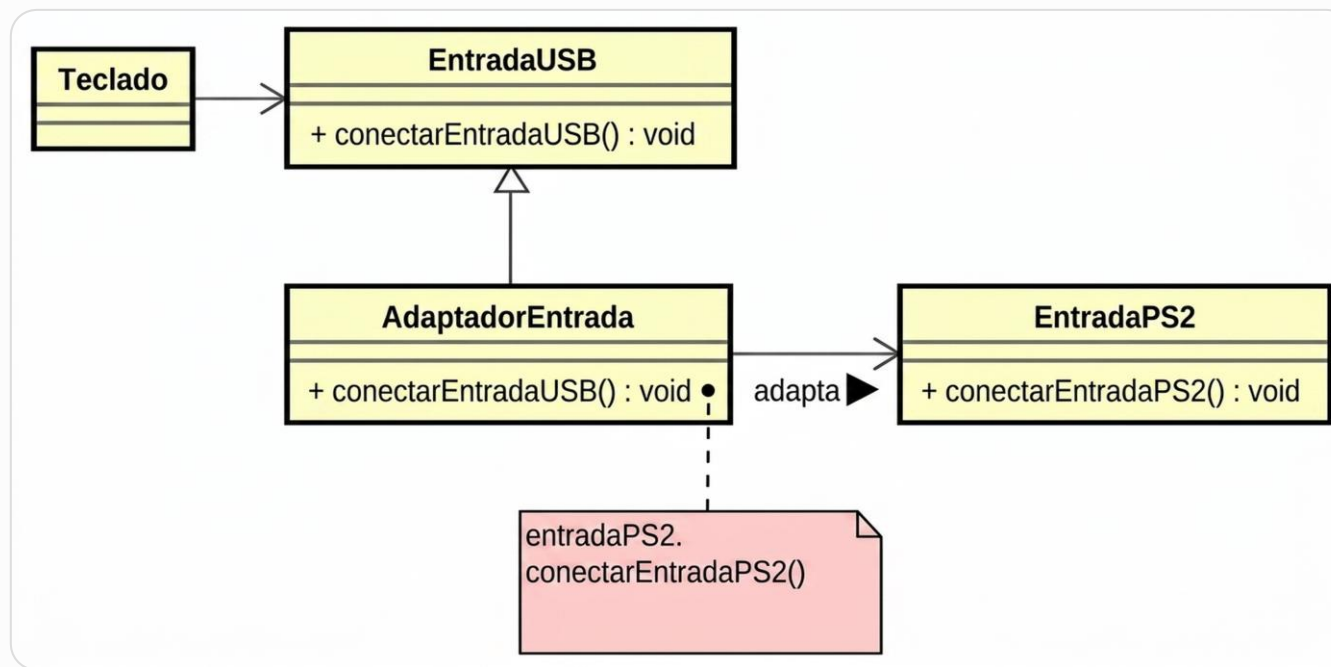


Adaptador en el ámbito de objeto

(Shvets, Alexander, 2018)

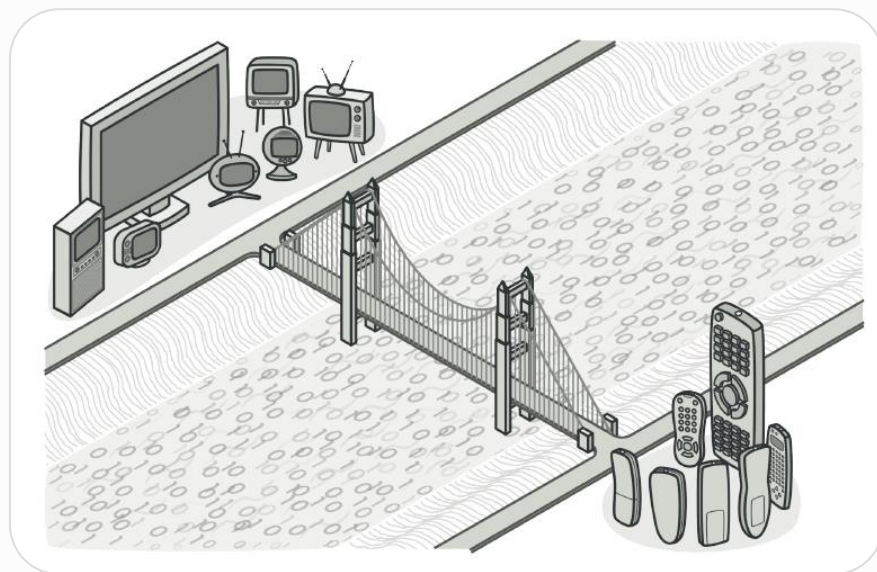
Patrón de Diseño Adaptador (del inglés Adapter)

- Ejemplo:

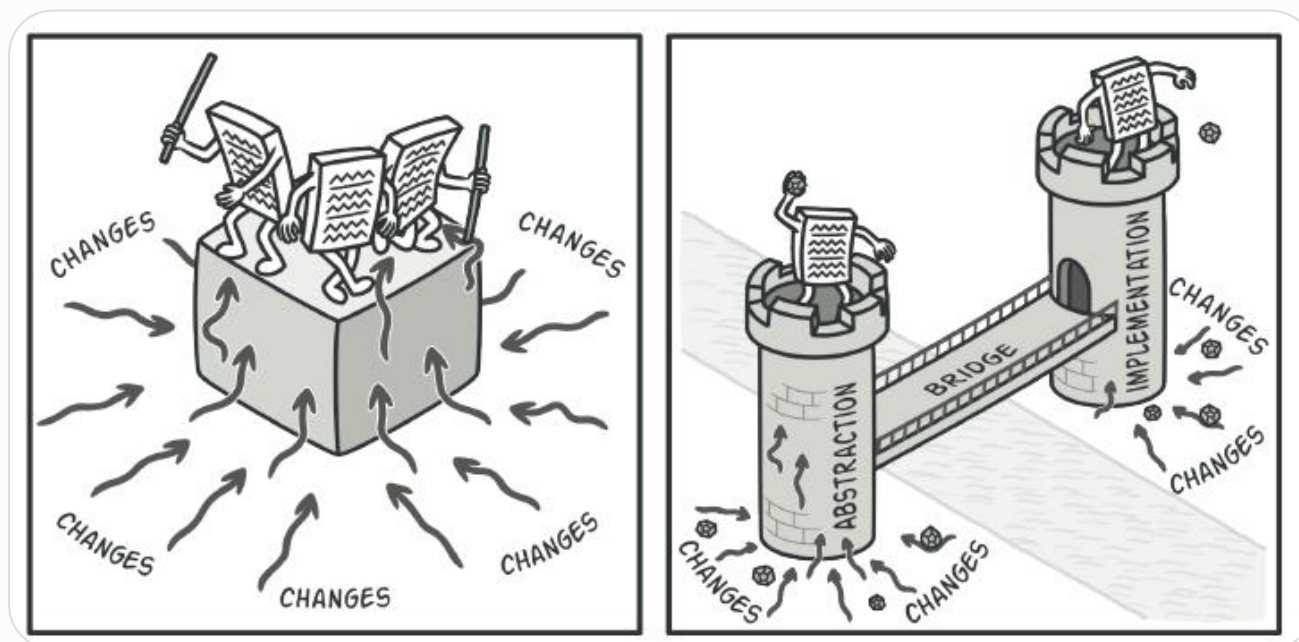


Patrón de Diseño Puente (del inglés Bridge)

- **Clasificación:** Patrón estructural/Ámbito de objetos
- **Intención:** Desacoplar una abstracción de su implementación para que ambas puedan variar independientemente.



(Shvets, Alexander, 2018)



Patrón de Diseño Puente

- Estructura:

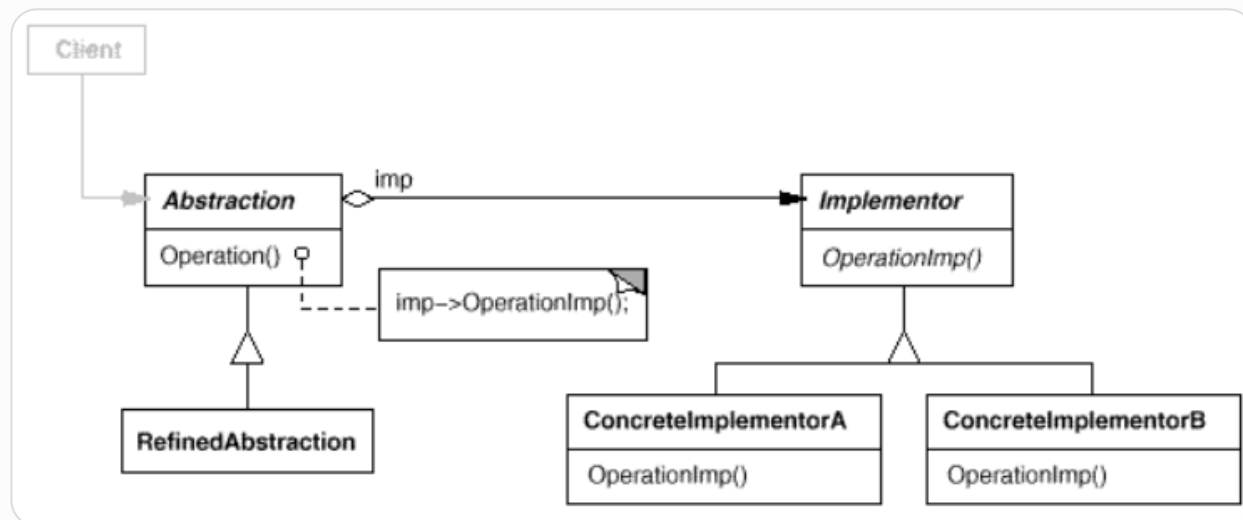
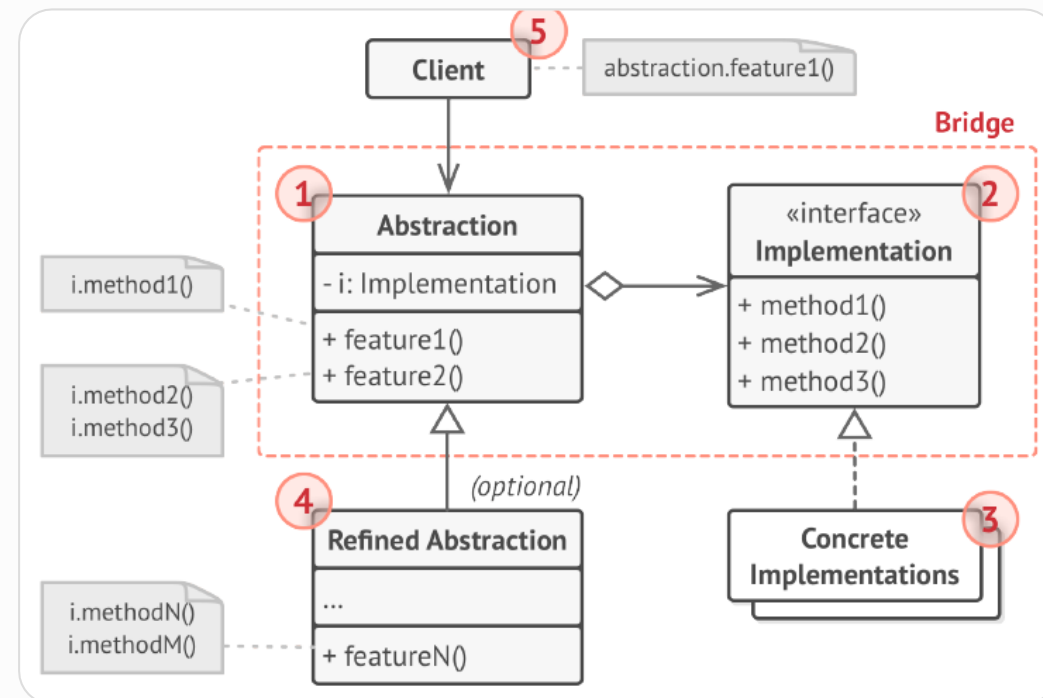


Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Próxima classe

- Patrones estructurales y de comportamiento

		Propósito		
		De creación	Estructurales	Conductual
Alcance	Clase	Método de fábrica	Adaptador	Intérprete Template Method
	Objeto	• Fábrica Abstracta • Constructor • Prototipo • Objeto Unitario (Singleton)	• Adaptador • Puente • Compuesto • Decorador • Fachada • Peso Ligero • Proxy	• Cadena de Responsabilidad • Comando • Iterador • Mediador • Memento • Observador • Estado • Estrategia • Visitador

Referencias

- Biggerstaff, Ted J. ; Perlis, Alan J. **Software Reusability: Concepts and Models**. Acm Press Frontier Series, 1989
- Alexander et al. **A Pattern Language**. New York: Oxford University Press, 1977.
- Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. **Design Patterns - Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1995
- Meszaros, G.; Doble, J. **A pattern language for pattern writing**, cap. 29 in J. Coplien; D. Schmidt. Pattern Languages of Program Design, Reading-MA, Addison-Wesley, p. 529–574, 1998.
- Buschmann F. et. al. **Pattern-oriented software architecture - a system of patterns**. Wiley, 1996.
- Coplien, J. Advanced C programming styles and idioms. Reading-MA: Addison-Wesley, 1992.
- Shvets, Alexander. **Dive into design patterns**, Refactoring.Guru, 2018, disponible para descargar en: <https://www.goodreads.com/book/show/43125355-dive-into-design-patterns>
- Braude, Eric. **Diseño de Software: De la programación a la arquitectura: Un enfoque basado en Java**, Bookman, 2005.

Lecturas recomendadas

- El sitio web <https://refactoring.guru/design-patterns> tiene material interesante sobre patrones, incluyendo ejemplos implementados en diferentes lenguajes de programación. Uno de los libros que utilicé en la composición de las diapositivas está disponible en línea:
 - Dive into design patterns - <https://refactoring.guru/design-patterns/catalog>
- El libro Gamma et al, 1995 fue utilizado como base para la preparación de las clases. Tenemos un ejemplar en la biblioteca del ICMC-USP e IME-USP.
- El libro Braude, 2005 también fue utilizado, con un menor número de referencias.
- El sitio web <https://hillside.net> contiene un excelente material sobre patrones de software en general, en particular Design Patterns. Contiene los artículos publicados en todas las conferencias sobre patrones realizadas en Estados Unidos y en varios lugares del mundo, incluido Brasil (SugarLoafPLoP). [The Hillside Group - A group dedicated to design patterns. Home of the patterns library.](#)

Para reflexionar para la próxima clase

- ¿Cuál es el impacto de la Inteligencia artificial en el desarrollo de software usando patrones de diseño?

¡Gracias!

Profa. Dra. Rosana T. Vaccare Braga | www.linkedin.com/in/rosana-braga-13778912

MBA **USP**
ESALQ